# Year Up United
# Data Analyst Training Academy: SQL Data Fundamentals

# Week 2 Student Guide

This courseware is to be distributed to accompany instructor led training.

Copyright © 2026 Pluralsight & The Reliable Collaboration Company

# Table of Contents

*Table of Contents* ..... 2

**MODULE 1 INTRODUCTION TO RELATIONAL DATABASES AND SQL ..... 5**

SECTION 1.A INTRODUCTION TO DATABASES ..... 6

*1.A.1 Week 2 Learning Objectives* ..... 7
*1.A.2 Week 2 Learning Outcomes* ..... 8
*1.A.3 What Are Databases?* ..... 9
*1.A.4 Overview of Databases* ..... 10
*1.A.5 Client-Server Model* ..... 11
*1.A.6 Database Operations* ..... 12
*1.A.7 Database Management Systems* ..... 13
*1.A.8 History of Relational Databases* ..... 14
*1.A.9 Relational Database Management Systems* ..... 15
*1.A.10 Components of a Database* ..... 16
*1.A.11 Schema* ..... 17
*1.A.12 Tables* ..... 18
*1.A.13 Primary and foreign keys* ..... 19
*1.A.14 Columns* ..... 20
*1.A.15 Rows* ..... 21
*1.A.16 Views* ..... 22

SECTION 1.B INTRODUCTION TO STRUCTURED QUERY LANGUAGE (SQL) ..... 23

*1.B.1 Introducing SQL* ..... 24
*1.B.2 A Little SQL History* ..... 25
*1.B.3 The `SELECT` Statement* ..... 26
*1.B.4 SQL Language Categories* ..... 27
*1.B.5 Syntax Categories* ..... 28
*1.B.6 SQL Language Rules* ..... 29
*1.B.7 Database Case Sensitivity* ..... 30
*1.B.8 Comments in SQL* ..... 31
*1.B.9 SQL Code Editors* ..... 32

SECTION 1.C WORKING WITH MYSQL WORKBENCH ..... 33

*1.C.1 Overview of MySQL Workbench* ..... 34
*1.C.2 Using MySQL Workbench* ..... 35
*1.C.3 Navigator & Information* ..... 36
*1.C.4 Query Editor and Output* ..... 37
*1.C.5 Writing SQL Queries in Workbench* ..... 38
*1.C.6 Executing SQL Queries in Workbench* ..... 39
*1.C.7 Displaying Query Results* ..... 40
*1.C.8 Exploring Schemas with Navigator* ..... 41
*1.C.9 Reviewing and Updating Preferences* ..... 42

**MODULE 2 RELATIONAL DATABASE DESIGN ..... 43**

SECTION 2.A UNDERSTANDING NORMALIZATION ..... 44

*2.A.1 Normalization* ..... 45
*2.A.2 Normalizing a Data Structure* ..... 46

*2.A.3 First Normal Form (1NF) ... 47*
*2.A.4 Second Normal Form (2NF) ... 48*
*2.A.5 Second Normal Form, continued ... 49*
*2.A.6 Third Normal Form (3NF) ... 50*
*2.A.7 Adding Unique Row Identifiers ... 51*
*2.A.8 Responsible Denormalization ... 52*

SECTION 2.B DATABASE DESIGN PROCESS ... 53

*2.B.1 Database Design Process ... 54*
*2.B.2 Model and Schema ... 55*
*2.B.3 Creating the Conceptual Model ... 56*
*2.B.4 Creating the Logical Model ... 57*
*2.B.5 Relationship Considerations ... 58*
*2.B.6 Resolving Many-to-Many Relationships ... 59*
*2.B.7 Documenting the Logical Model ... 60*
*2.B.8 ER Diagram Tools ... 61*
*2.B.9 Creating the Physical Model ... 62*
*2.B.10 Physical Database Files ... 63*
*2.B.11 Review of the Design Process ... 64*

**MODULE 3 WORKING WITH TABLES ... 65**

SECTION 3.A DESIGNING TABLES ... 66

*3.A.1 Choosing a Primary Key ... 67*
*3.A.2 Foreign Keys ... 68*
*3.A.3 Relationships: Cardinality ... 69*
*3.A.4 Relationships: Existence ... 70*
*3.A.5 Using Recursive Relationships ... 71*
*3.A.6 Defining Columns ... 72*
*3.A.7 More on Data Types ... 73*
*3.A.8 String Data Types ... 74*
*3.A.9 String Data Types in MySQL ... 75*
*3.A.10 Exact Numeric Data Types ... 76*
*3.A.11 Approximate Numeric Data Types ... 77*
*3.A.12 So… What is an Approximate Value?? ... 78*
*3.A.13 Examples of Approximate Values ... 79*
*3.A.14 Boolean Type ... 80*
*3.A.15 Datetime Data Types ... 81*
*3.A.16 Unique Identifiers ... 82*
*3.A.17 Other Data Types ... 83*
*3.A.18 `NULL` Values ... 84*

SECTION 3.B CREATING AND MODIFYING TABLES ... 85

*3.B.1 Data Definition Language (DDL) ... 86*
*3.B.2 Creating a Table ... 87*
*3.B.3 Modifying a Table ... 88*
*3.B.4 Using a Default Value ... 89*
*3.B.5 Constraints ... 90*
*3.B.6 Constraint Names ... 91*
*3.B.7 Primary Key / Foreign Key Constraints ... 92*
*3.B.8 Using Auto-increment or Identity Columns ... 93*
*3.B.9 `ON DELETE` AND `ON UPDATE` Constraints ... 94*
*3.B.10 `Unique` Constraint ... 95*

*3.B.11 `CHECK` Constraints* ..... *96*
*3.B.12 Example: Check Constraints* ..... *97*
*3.B.13 Dropping a Constraint* ..... *98*
*3.B.14 Creating a Temporary Table* ..... *99*
*3.B.15 Dropping a Table* ..... *100*

**MODULE 4 WORKING WITH RECORDS ..... 101**

SECTION 4.A INSERTING, UPDATING, AND DELETING RECORDS ..... 102
*4.A.1 Introducing DML* ..... *103*
*4.A.2 `INSERT` Statements* ..... *104*
*4.A.3 `UPDATE` Statements* ..... *105*
*4.A.4 DELETE Statements* ..... *106*
*4.A.5 `SELECT` Statements* ..... *107*
*4.A.6 `WHERE` Clauses* ..... *108*
SECTION 4.B ERROR HANDLING ..... 109
*4.B.1 Brief Intro to Error Handling Concepts* ..... *110*
*4.B.2 Testing for Errors with SELECT* ..... *111*
*4.B.3 Using Transactions* ..... *112*
SECTION 4.C BATCH IMPORTING AND EXPORTING ..... 113
*4.C.1 Importing existing data* ..... *114*
*4.C.2 Exporting table data* ..... *115*
*4.C.3 Importing an existing database* ..... *116*

# Module 1

# Introduction to Relational Databases and SQL

# Section 1.A

# Introduction to Databases

# 1.A.1 Week 2 Learning Objectives

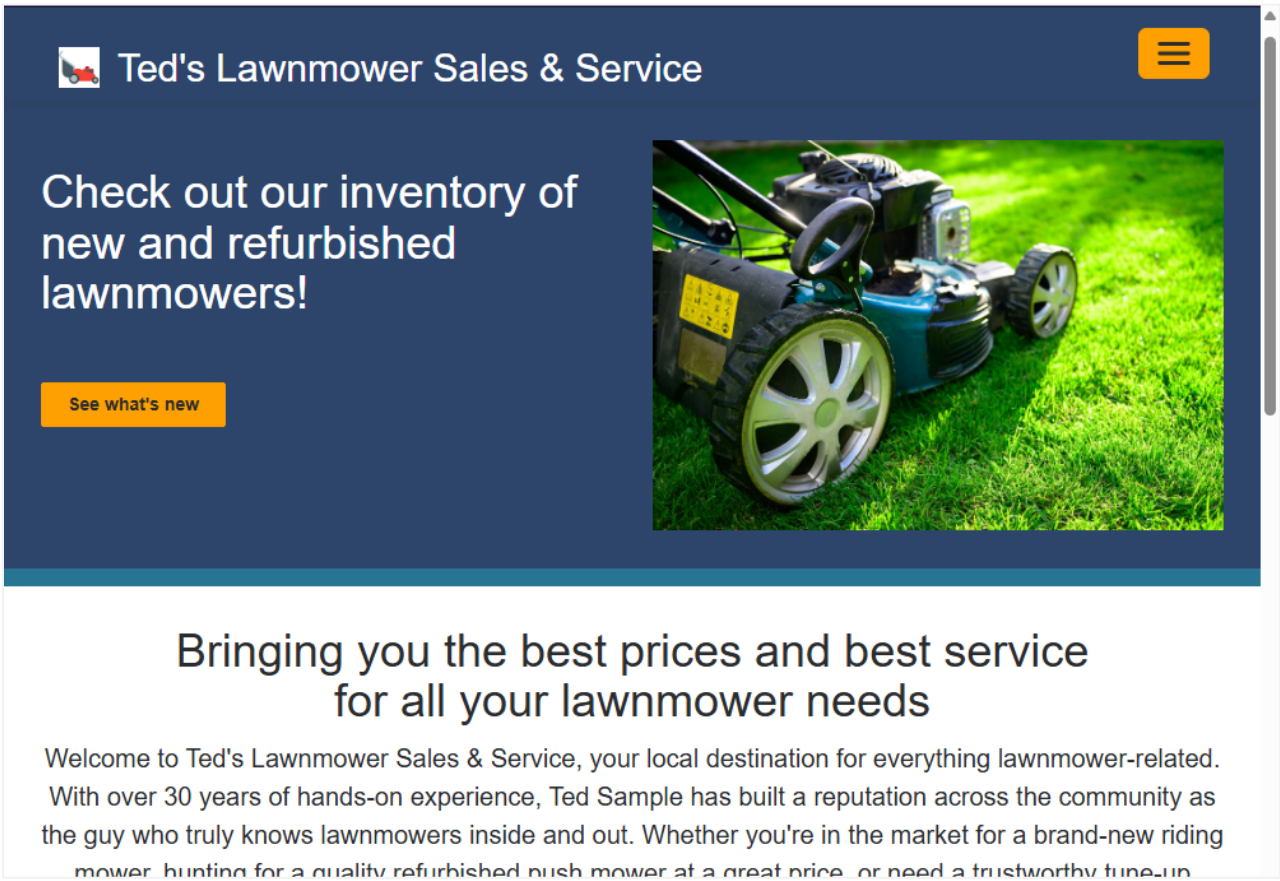
- This week you will learn:
  - ◊ Relational database concepts: tables, schemas, keys, relationships, and views
  - ◊ The architecture of relational database management systems (RDBMS) and how they are used in organizations
  - ◊ Introduction to SQL and its major language categories (DDL, DML, DCL)
  - ◊ Using MySQL Workbench to write, execute, and save SQL queries
  - ◊ Database normalization principles (1NF, 2NF, 3NF) and the database design process
  - ◊ Conceptual, logical, and physical data modeling; entity-relationship (ER) diagrams
  - ◊ Table design: choosing appropriate data types, defining keys, and establishing relationships
  - ◊ Data Definition Language (DDL): creating, modifying, and dropping tables and constraints
  - ◊ Data Manipulation Language (DML): inserting, updating, and deleting records
  - ◊ Importing and exporting data between MySQL and external file formats
  - ◊ Version control with git and GitHub for managing and sharing SQL scripts

# 1.A.2 Week 2 Learning Outcomes

- By the end of this week, you will be able to:
  - ◊ Describe the components of a relational database (tables, rows, columns, primary/foreign keys, views) and explain how they relate to one another
  - ◊ Write basic SELECT queries to retrieve and review data from a database in MySQL Workbench
  - ◊ Apply normalization rules (1NF, 2NF, 3NF) to evaluate and restructure a data model
  - ◊ Design a relational database by creating a conceptual model and identifying appropriate tables and relationships
  - ◊ Choose appropriate data types for table columns based on the nature of the data
  - ◊ Write DDL statements to create, modify, and drop tables with appropriate constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT)
  - ◊ Write DML statements to insert, update, and delete records
  - ◊ Import a CSV file into MySQL and export query results to CSV and JSON formats
  - ◊ Use git and GitHub to save, commit, and share SQL scripts

# 1.A.3 What Are Databases?

- **Databases** are collections of organized data
  - ◊ Could be created to support an application, website, or other tool/utility
  - ◊ Data is created *by* and *for* something – it never exists in a vacuum
- As discussed in Week 1, data can be collected in many different ways and can consist of many different types of information – so a database is a means for holding and organizing that information
- Databases are commonly used wherever data needs to be collected and managed
  - ◊ For example, what kind of data might need to be managed here?



# 1.A.4 Overview of Databases

- Fundamentally a database is simply an organized collection of information

- Modern databases have these common characteristics:
  - ◊ Capacity sufficient to store the data
  - ◊ Security and auditing – guarding against accidental "big oops" data errors as well as malicious attacks
  - ◊ Multiuser accessibility
  - ◊ Searchability
  - ◊ Scalability – that is, ideally databases should be adaptable to changing business needs such as permitting modification of DB objects or moving to new hardware/software
  - ◊ User friendliness
    - * Not all users are programmers or technically savvy, meaning that data needs to be easy to access for any user with via a graphical user interface that clearly defines a user's options and outputs results
    - * However, it's also generally desirable to have the database accessible for ad hoc queries and reports by an administrator or power user

# 1.A.5 Client-Server Model

- The client-server model is often used for database systems:
  - ◊ Databases reside on a central server, which enforces data security and integrity
  - ◊ Client applications are installed on end-user computers and connect to the database server, allowing multiple clients to connect to the database at the same time
- The **client application** installed on a user's own local computer provides a user interface to interact with the database
  - ◊ For example, software for an inventory control system can be installed on an individual user's computer
  - ◊ This inventory software connects to a database hosted on a server
  - ◊ As the user interacts with the software on their computer, the inventory data gets read from and stored back to the database on the server
- Technically, a **server** is a computer program that provides resources or functionality to other (**client**) programs
  - ◊ While commonly we refer to servers on external computing resources as "servers," it's also possible to have a server running on your own local computer (as we will see!)

# 1.A.6 Database Operations

- When we store data in a database, there are four basic types of operations we perform with data
  - ◊ Add new data
  - ◊ Retrieve the data for review or to answer questions
  - ◊ Change existing data
  - ◊ Delete data
- The common acronym for these four types of operations is **CRUD**
  - ◊ Create
  - ◊ Read
  - ◊ Update
  - ◊ Delete

# 1.A.7 Database Management Systems

- The structural framework of a database depends on the **database management system (DBMS)** used

- Modern DBMS fall into two broad categories:
  - ◊ **Relational databases** are SQL-based databases that manage their data in collections of related tables
  - ◊ "NoSQL" databases have an entirely different way of organizing data and providing access to it
    - ∗ **Graph databases** store nodes and relationships instead of tables or documents, similar to how you might sketch ideas on a whiteboard
    - ∗ **Document databases** store information as documents composed of field-value pairs, saved as JSON, XML, or another format

- Relational databases and **Relational Database Management Systems (RDBMS)** are still the most commonly used type of database, and those will be our focus in this course

# 1.A.8 History of Relational Databases

- The relational database model was developed in the 1970s by an IBM researcher, Dr. Edgar F. Codd

- Remember, in a relational database, the data is organized in related tables made up of rows and columns
  - ◊ Each row represents one set of data, often also called a record or entry
  - ◊ Each column represents a field, a piece of information within that record

- This “new” model provided a new way of managing issues often faced with standard bulk file storage
  - ◊ A well-designed relational database reduces data redundancy, saving disk space and providing more efficient retrieval of data
  - ◊ Security can be more easily defined for each database, e.g. managing permissions for which users can connect, scheduling backup routines and maintenance activities
  - ◊ Places management of the physical location of data at the database level, allowing for storage across multiple disks and/or partitions to improve performance

# 1.A.9 Relational Database Management Systems

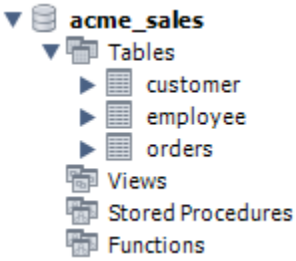
- An RDBMS allows an administrator to:
  - ◊ Store billions of records efficiently in data tables using a relational hierarchy in which related tables can be queried together to provide useful information
  - ◊ Execute queries in a standardized language (SQL) using a code editor
  - ◊ Create and manage multiple databases on a server
  - ◊ Manage multiple database servers on a network
  - ◊ Implement backup/restore protocols for databases
  - ◊ Exercise security controls over which users can read/write data in each table
  - ◊ Use graphical tools to manage database and database server configuration settings
- A few popular RDBMS:
  - ◊ MySQL (Open Source)
  - ◊ Microsoft SQL Server (Commercial)
  - ◊ PostgreSQL (Open Source)
- Different systems may implement slightly different versions of SQL (e.g. use of different keyword commands)

# 1.A.10 Components of a Database

- In a relational database, the data is organized in related **tables** made up of rows and columns
- Each row represents one set of data and is often called a **record** or **entry** (or you might just call it a row)
- Each **column** represents a particular property or attribute of that record, also often called a **field**
- The **schema** is the overall roadmap for what tables exist and how they relate to each other

# 1.A.11 Schema

- The schema defines the overall shape for your database such as what tables it includes
  - ◊ For example, a business might have a table for customer information, another for employee information, and an Orders table that references both Customer and Employee




    - * The schema may also contain other features like indexes, views, and stored procedures; for purposes of this course, we will not dive deeply into these other database concepts

- You might think of a schema as a well-organized closet

# 1.A.12 Tables

- A table contains information for a particular sort of thing
  - ◊ For example, the Orders table might include the date an order was made, by whom, for how much, and who was the employee who took the order – but to find the contact info for the customer, or contact info for the employee, you would need to look at the Customer and Employee tables
- Again, tables are made up of rows and columns
- Rows = records or entries
  - ◊ Each row represents one set of information, e.g. info about one customer, one order, or one employee
- Columns = fields or attributes or properties
  - ◊ Each column contains a category of information across all records
  - ◊ Examples:
    - ∗ Customer: customer id, business name, contact name, email, phone, market segment
    - ∗ Employee: employee id, name, title, email, phone, sales region
    - ∗ Orders: date placed, customer id, employee id, total purchase price, sales tax, total amount due
- In the closet analogy: a table is like a particular set of shelves or area of the closet which contains things all of the same kind
  - ◊ When you put together an outfit, you usually need to pull together multiple items from different areas of the closet 👕 + 👖 + 🧦

# 1.A.13 Primary and foreign keys

- Each table must have a column that uniquely identifies each row in the table, called a **primary key**
  - Example:
    - In the Customer table, the column for “Customer ID” is primary key
    - In the Orders table, the column for “order id” is primary key
- Relationships can be drawn between tables by using the primary key of one table as a **foreign key** in another related table
  - Example:
    - In the Orders table, the column for “Customer ID” is a foreign key that points back to the Customer table
- Analogy: You might think of key columns as a way that a record from one table @ tags a record in another table
  - In this analogy, the primary key could be a list of Instagram handles and the foreign key would be where you tag someone in an Instagram post

# 1.A.14 Columns

- Columns define the shape and type of the data in a table

- Columns must specify a:
  - ◊ **name**
  - ◊ **datatype** (e.g. text/VARCHAR, number, Boolean, or other type)
    - ∗ A datatype may or may not have a precision, e.g. number of characters
    - ∗ Some datatypes take up less memory than others (e.g. more memory needed to store alphanumeric text than numbers)

| | au_id | au_lname | au_fname | phone | address | city | state | zip | contract |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 172-32-1176 | White | Johnson | 408 496-7223 | 10932 Bigge Rd. | Menlo Park | CA | 94025 | 1 |
| 2 | 213-46-8915 | Green | Marjorie | 415 986-7020 | 309 63rd St. #411 | Oakland | CA | 94618 | 1 |
| 3 | 238-95-7766 | Carson | Cheryl | 415 548-7723 | 589 Darwin Ln. | Berkeley | CA | 94705 | 1 |
| 4 | 267-41-2394 | O'Leary | Michael | 408 286-2428 | 22 Cleveland Av. #14 | San Jose | CA | 95128 | 1 |
| 5 | 274-80-9391 | Straight | Dean | 415 834-2919 | 5420 College Av. | Oakland | CA | 94609 | 1 |
| 6 | 341-22-1782 | Smith | Meander | 913 843-0462 | 10 Mississippi Dr. | Lawrence | KS | 66044 | 0 |
| 7 | 409-56-7008 | Bennet | Abraham | 415 658-9932 | 6223 Bateman St. | Berkeley | CA | 94705 | 1 |
| 8 | 427-17-2319 | Dull | Ann | 415 836-7128 | 3410 Blonde St. | Palo Alto | CA | 94301 | 1 |
| 9 | 472-27-2349 | Gringlesby | Burt | 707 938-6445 | PO Box 792 | Covelo | CA | 95428 | 1 |
| 10 | 486-29-1786 | Locksley | Charlene | 415 585-4620 | 18 Broadway Av. | San Francisco | CA | 94130 | 1 |
| 11 | 527-72-3246 | Greene | Morningstar | 615 297-2723 | 22 Graybar House Rd. | Nashville | TN | 37215 | 0 |

- Columns can be configured to allow (or *not* allow) `NULL` values
  - ◊ In SQL, `NULL` is not the same thing as zero or blank
    - ∗ A value of zero (0) means that value, 0, exists in the record
    - ∗ A blank value means that for that item, a blank entry was recorded
    - ∗ `NULL` is a special marker used in SQL to indicate that *no record exists*

# 1.A.15 Rows

- A **row** is a **record** of data within a table

- The record is made up of the values stored in each column

| | au_id | au_lname | au_fname | phone | address | city | state | zip | contract |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 172-32-1176 | White | Johnson | 408 496-7223 | 10932 Bigge Rd. | Menlo Park | CA | 94025 | 1 |
| 2 | 213-46-8915 | Green | Marjorie | 415 986-7020 | 309 63rd St. #411 | Oakland | CA | 94618 | 1 |
| 3 | 238-95-7766 | Carson | Cheryl | 415 548-7723 | 589 Darwin Ln. | Berkeley | CA | 94705 | 1 |
| 4 | 267-41-2394 | O'Leary | Michael | 408 286-2428 | 22 Cleveland Av. #14 | San Jose | CA | 95128 | 1 |
| 5 | 274-80-9391 | Straight | Dean | 415 834-2919 | 5420 College Av. | Oakland | CA | 94609 | 1 |
| 6 | 341-22-1782 | Smith | Meander | 913 843-0462 | 10 Mississippi Dr. | Lawrence | KS | 66044 | 0 |
| 7 | 409-56-7008 | Bennet | Abraham | 415 658-9932 | 6223 Bateman St. | Berkeley | CA | 94705 | 1 |
| 8 | 427-17-2319 | Dull | Ann | 415 836-7128 | 3410 Blonde St. | Palo Alto | CA | 94301 | 1 |

- A table could store zero records, 10 records, 10 million records, maybe even billions of records!

- When you were using Excel, you might have identified a row in a spreadsheet by its row number – but databases don't work that way
  - ◊ A record does not have a fixed place within a table – there's no such thing as a "row number" like in a spreadsheet
  - ◊ Instead, a record is typically identified by its unique value stored in the primary key column
    - ∗ In the screenshot above, the primary key column is au_id (short for "author id")
  - ◊ When retrieving records from a table, by default they are returned in the order that is easiest from the computer's perspective based on how they are stored in memory
    - ∗ That *might* correspond to the primary key column, but not necessarily!

# 1.A.16 Views

- A **view** is a stored query that is saved in the database and can be used like a table
  - ◊ Unlike a table, a view does not store data itself — it simply stores the query definition, and the underlying data is retrieved from the base tables each time the view is queried
  - ◊ Views are sometimes called "virtual tables" for this reason
- Views are useful for several purposes:
  - ◊ Simplifying complex queries — a frequently used multi-table join or complex SELECT can be saved as a view, allowing users to query it like a simple table
  - ◊ Restricting access to data — a view can expose only certain columns or rows from a table, allowing administrators to control what data different users can see without changing the underlying table structure
  - ◊ Presenting data in a more readable or useful format — for instance, a view might combine first and last name columns into a single FullName field, or apply a CASE structure to translate coded values into readable labels
- We will talk about views more next week

# Section 1.B

# Introduction to Structured Query Language (SQL)

# 1.B.1 Introducing SQL

- Structured Query Language (SQL) is the standard language used to communicate with SQL-based databases
  - ◊ SQL is the first of four languages that we will learn during this academy!
  - ◊ Fun Fact: Programmers can be opinionated about how to pronounce SQL
    - ∗ The official standard says it's S-Q-L, but many people including top experts who have written books on the subject pronounce it "sequel"
- **Queries** can include both asking questions of your data, but also writing commands to create or alter the data
  - ◊ Each request sent to the database server is considered a query
  - ◊ Each request will receive a response
    - ∗ May be as simple as "Success!"
    - ∗ May contain a tabular data stream as a result
- Each RDBMS comes with developer tooling for creating queries, often through a graphical user interface (GUI)
- Vendors tend to deviate from the standard with extra features or functions
  - ◊ These product specific languages are considered **SQL dialects**
  - ◊ We will stick to standard SQL wherever possible in this course, but will favor the MySQL dialect where needed

# 1.B.2 A Little SQL History

- SQL got its start in the 1970s as the programming language used to interface with the first relational databases
  - ◊ As relational databases grew and evolved, so did SQL – sometimes along very distinct branches depending on the DBMS
- Starting in 1986, the American National Standards Institute (ANSI) began releasing standards for SQL
  - ◊ These SQL standards have since been updated multiple times, in 1989, 1992, 1999, 2003, 2006, 2008, 2011, 2016 and most recently, 2023
  - ◊ Starting in 1989, the International Standards Organization (ISO) also joined in setting these standards
- Today all major RDBMS platforms generally comply with ANSI standards
  - ◊ Vendors can still create their own proprietary dialect by adding unique functionality not addressed in the SQL standards
  - ◊ Vendors may also add variations to the ANSI standard or may choose not to implement part of the standard
  - ◊ For the most part, though, SQL code is sufficiently standard to be portable across platforms

# 1.B.3 The **SELECT** Statement

- We'll dive deeper into querying data later, but the first starting place is the `SELECT` statement

- Use **`SELECT`** to retrieve all or selected columns from a table, plus keyword **`FROM`** to identify the specific table
  - ◊ If multiple columns are used, separate them with a comma
  - ◊ To retrieve *all* columns, use an asterisk (*)

- Writing a query with `SELECT`:
  - ◊ The basic syntax of a `SELECT` statement is

```
SELECT [columns] FROM table_name;
```

  - ◊ Examples:

```
SELECT * FROM sakila.actor;

SELECT title, description FROM sakila.film;
```

    - ∗ Notice that a SQL query always concludes with a semicolon (;)
    - ∗ Spaces and line spaces are not significant, so the above query could also be written as

```
SELECT title, description
FROM sakila.film;
```

# 1.B.4 SQL Language Categories

- In learning more about SQL, you may encounter the following categories used to refer to different types of queries:
  - ◊ Data Definition Language (DDL)
    - ∗ Used to define databases and database objects
    - ∗ E.g. creating databases, creating or modifying a view
    - ∗ Keywords: `CREATE`, `ALTER`, `DROP`
  - ◊ Data Manipulation Language (DML)
    - ∗ Used to retrieve and/or modify records
    - ∗ E.g. deleting a record, updating a record
    - ∗ Keywords: `INSERT`, `UPDATE`, `DELETE`, `SELECT`
  - ◊ Data Control Language (DCL)
    - ∗ Sets permissions on database objects
    - ∗ E.g. allowing a specified user to update records in a table
    - ∗ Keywords: `GRANT`, `DENY`, `REVOKE`
      *(MySQL does not use `DENY` but it exists in other SQL dialects, specifically T-SQL for Microsoft SQL Server)*

- This class will dive most deeply into DML, but we'll kick of this week working with DDL

# 1.B.5 Syntax Categories

- SQL statements, assembled into queries, are instructions sent to the RDBMS for processing

- The syntax making up SQL statements include the following:
  - ◊ **Keywords** are the instructions telling the RDBMS what to do
    - * E.g. `SELECT` is used to retrieve records
  - ◊ **Identifiers** are names of databases and their objects
    - * E.g. a table might be named **`Products`**
  - ◊ **Literals** are hard-coded values, typically strings (text), numbers, or dates
    - * E.g. **`'hello'`** or **`3.14`** or **`'03-14-2020'`**
  - ◊ **Operators** are symbols used to perform mathematical operations or to compare data

    `+`, `-`, `*`, and `/` are mathematical operators

    =, >, <, >=, and <= are comparison operators

| |
|---|
| If you need a memory cue to help tell apart the greater-than and less-than symbols, remember that the heart emoji **<3** is **"less than 3"** |

# 1.B.6 SQL Language Rules

- Keywords are not case sensitive (but upper case is often used), e.g. `SELECT` or `select` work just the same

- As a best practice, identifiers usually do not contain spaces
  - ◊ Example: `order_details` or `OrderDetails` are both acceptable
  - ◊ If an identifier has a space in its name, in MySQL it must be surrounded by backticks when used in a query, e.g. `` `Order Details` ``

- Text (string) values must be inside single (`' '`) or double quotes (`" "`)
  - ◊ Example: `'Hello World!'` or `"Hello World!"` both work
  - ◊ But beware when using copy-paste, *not* curly quotes (“Hello world!”)

- Date values can be coded as strings in the form 'YYYY-MM-DD'
  - ◊ Example: `'2024-07-02'`

- Numeric values do *not* use quotes
  - ◊ Example: `42, 3.14`

- White space is ignored (e.g. extra spaces, tabs, line breaks)
  - ◊ Use this to your advantage to create easy-to-read queries!

- Each SQL statement should end in a semicolon (`;`)
  - ◊ MySQL does not strictly require this, but ANSI standard does

# 1.B.7 Database Case Sensitivity

- Case sensitivity refers to whether capital letters and lowercase letters are treated as mutually equivalent or distinct ("`A = a`"?)

- Database objects in MySQL are commonly *not* case sensitive but this depends on the settings for the database
  - ◊ Database collation settings specify rules for how strings are sorted and compared, and can be set to apply case sensitivity
  - ◊ Example: Where case sensitivity applies, querying `'Person'` will not return `'PERSON'` or `'person'`
    - * However, note that no two objects (like tables, or columns in a table) can have the same name, even if they use different cases

- Defaults for case sensitivity depend on the DBMS
  - ◊ Data in MySQL, SQL Server, and Db2 databases is typically *not* case sensitive, while data in an Oracle or PostgreSQL database usually is

- Reminder, SQL keywords in a query are **not** case sensitive
  - ◊ Example: `SELECT`, `Select`, and `select` all work the same
  - ◊ It's conventional to see these in all upper case, and using all caps for commands can make your code easier to read ...
  - ◊ ... but you will also find reference materials consistently use all lower case for example SQL queries, and that's just fine too

# 1.B.8 Comments in SQL

- A **comment** is text that is included for human readers of the code but ignored by the computer

- For single-line comments, begin the comment with -- or #

```
-- This is a valid comment in SQL
# This is also a valid comment
```

- A single line comment can also follow a SQL statement inline

```
SELECT * FROM customer; -- This is a comment
```

- Multi-line comments must start with /* and conclude with */

```
/* This is an example of a
valid multi-line comment */
```

- A multi-line comment can also be used inside an SQL statement

```
SELECT name, email /* This is a comment */ FROM
customer;
```

- In the MySQL Workbench code editor, comments show as light blue

# 1.B.9 SQL Code Editors

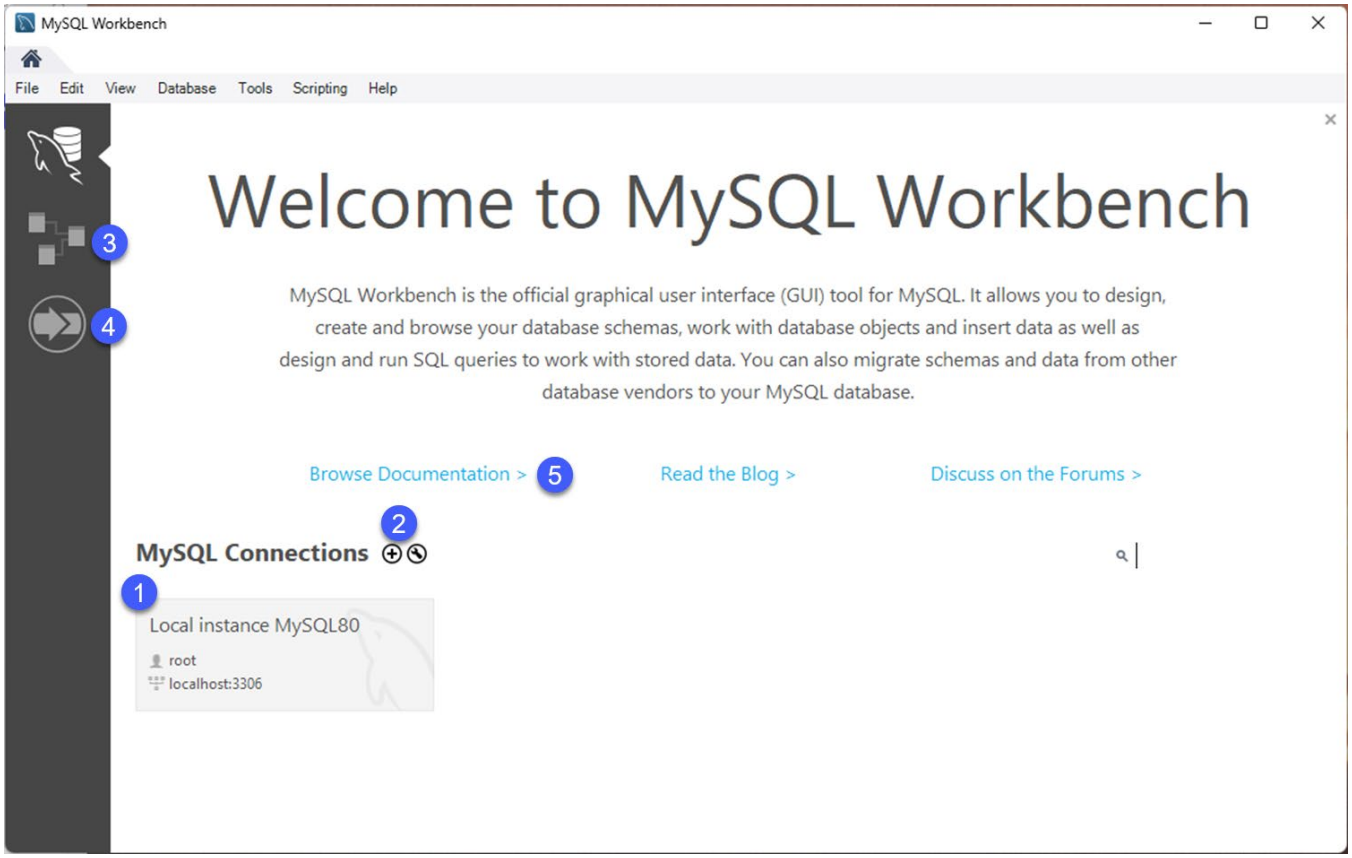
- Every RDBMS platform provides its own SQL code editor:
  - ◊ MySQL Workbench works with MySQL databases
  - ◊ SQL Server Management Studio works with SQL Server (Microsoft)
  - ◊ pgAdmin works with PostgreSQL databases
  - ◊ SQL Plus for Oracle
  - ◊ Third party tools also exist, such as Toad for Oracle
- The **code editor** is where SQL queries are created and run
- The code editor also displays the **result** of a query
  - ◊ If the query is executed successfully, a “success” message is returned
  - ◊ An error message is returned where there is a problem executing the query
- Some code editors are more of a graphical user interface (GUI), while others are just command line
  - ◊ MySQL Workbench is more of a GUI
  - ◊ It’s also possible to work with MySQL in the command line

# Section 1.C

# Working with MySQL Workbench

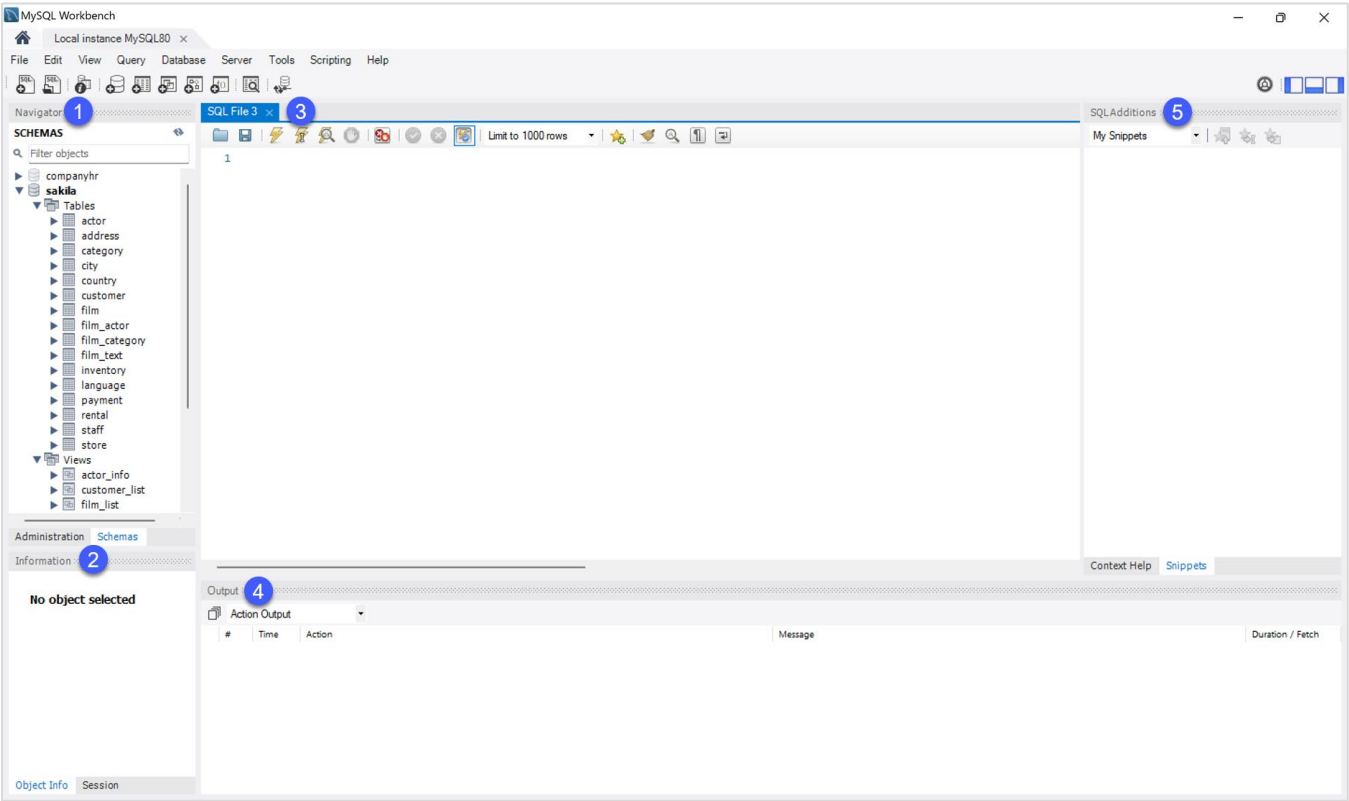
# 1.C.1 Overview of MySQL Workbench

- MySQL Workbench opens to a home screen with a few options:
    1. Connect to an existing MySQL connection
    2. Create or manage connections (using the + or wrench icons)
    3. Open, create, or manage data models
    4. Migrate an existing SQL database to MySQL
    5. Links to documentation and other information
- We will be working with the **Local instance** connection under MySQL Connections that was created on installation



# 1.C.2 Using MySQL Workbench

- Once connected to an instance, you'll see the workspace in MySQL Workbench is divided into multiple panes
    1. Navigator
    2. Information
    3. Query editor
    4. Output
    5. SQL Additions *(we will not use this feature in this course)*

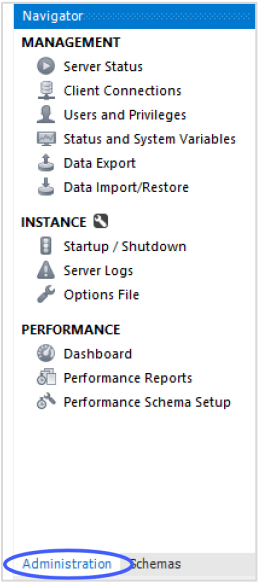


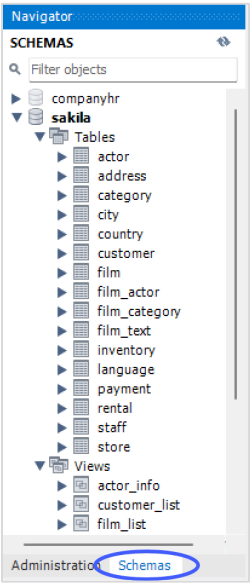

- There are also a number of toolbar buttons and file menu options that we will explore

# 1.C.3 Navigator & Information

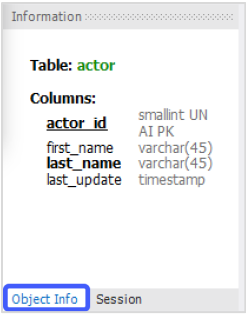
- The **Navigator** and **Information** panels are found on the left side of the workspace

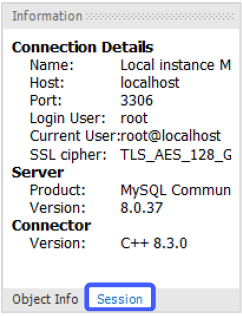
- Navigator has two available tabs:
  - ◊ **Administration** lets you perform administrative tasks
  - ◊ **Schemas** lets you view and work with your databases



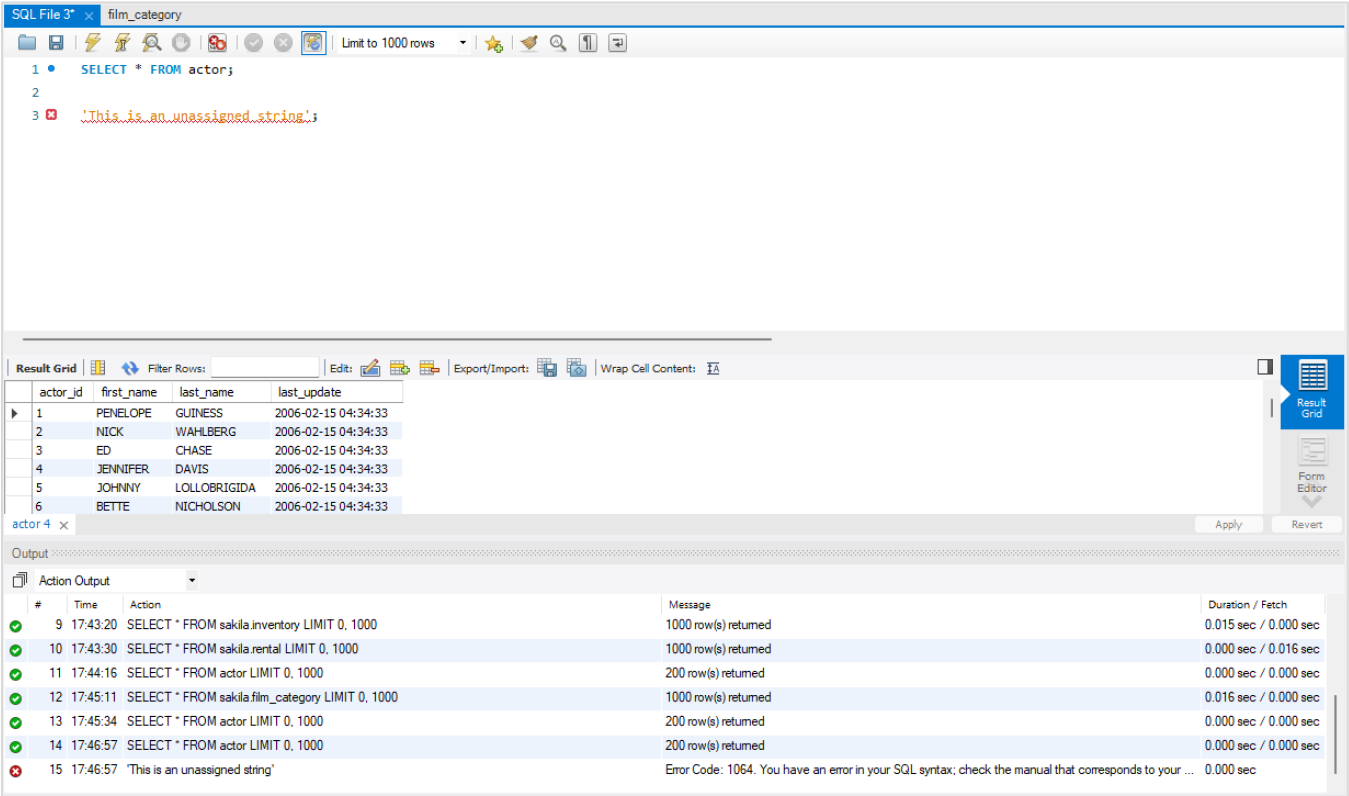

- Information displays **Object Info** about the most recently selected database object, and connection info for the **Session**



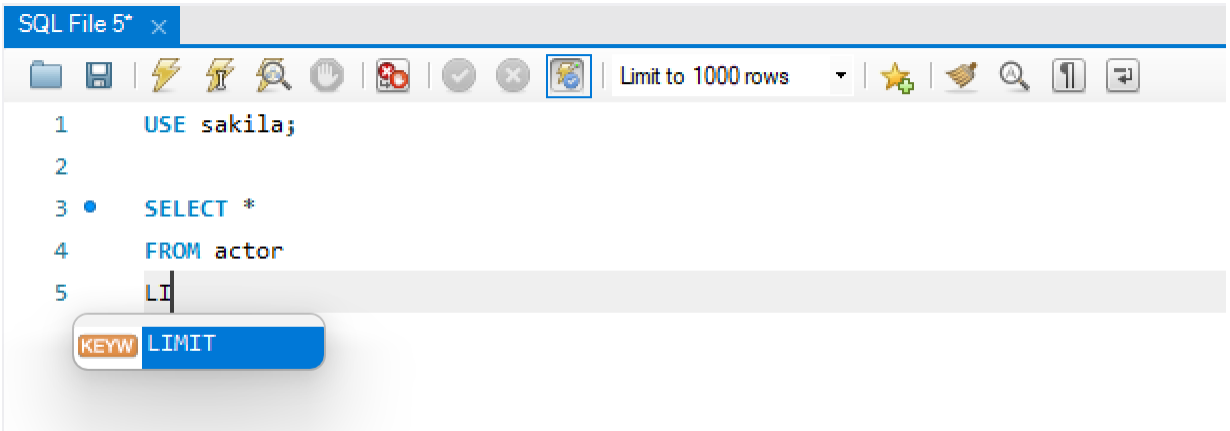
# 1.C.4 Query Editor and Output

- The **Query Editor** opens in the main central panel
  - ◊ Multiple SQL tabs can be open at the same time with different queries
  - ◊ Other utilities and reports also open as new tabs in this central panel
- **Output** is displayed at bottom and shows the results of each query that has been executed
  - ◊ If multiple query tabs are open, all output results will display together
- If a query returns records from one or more tables, these will display in a **Result Grid** that appears between the Query Editor and Output panels
  - ◊ From the Result Grid, results can be filtered, edited, or exported



# 1.C.5 Writing SQL Queries in Workbench

- To start writing SQL code from a blank file:
  - ◊ Click on the +SQL icon,
  - ◊ Or go to **File** > **New Query Tab**
- First you must select the database that you want to work with:
  - ◊ Double-click on the database in Navigator
  - ◊ Start your query with a `USE` statement including the database name
    - * Example: `USE example_database;`
  - ◊ Specify the database in front of the table name, joined by a period
    - * Example: `SELECT * FROM example_database.orders;`
- A few things to notice that as you begin typing a query
  - ◊ Keywords and comments are automatically displayed in blue text, while values such as strings and numbers are displayed in red
  - ◊ As you type, keywords and identifiers may be suggested in a dropdown; arrow up or down to make a selection, and hit tab for autocompletion



# 1.C.6 Executing SQL Queries in Workbench

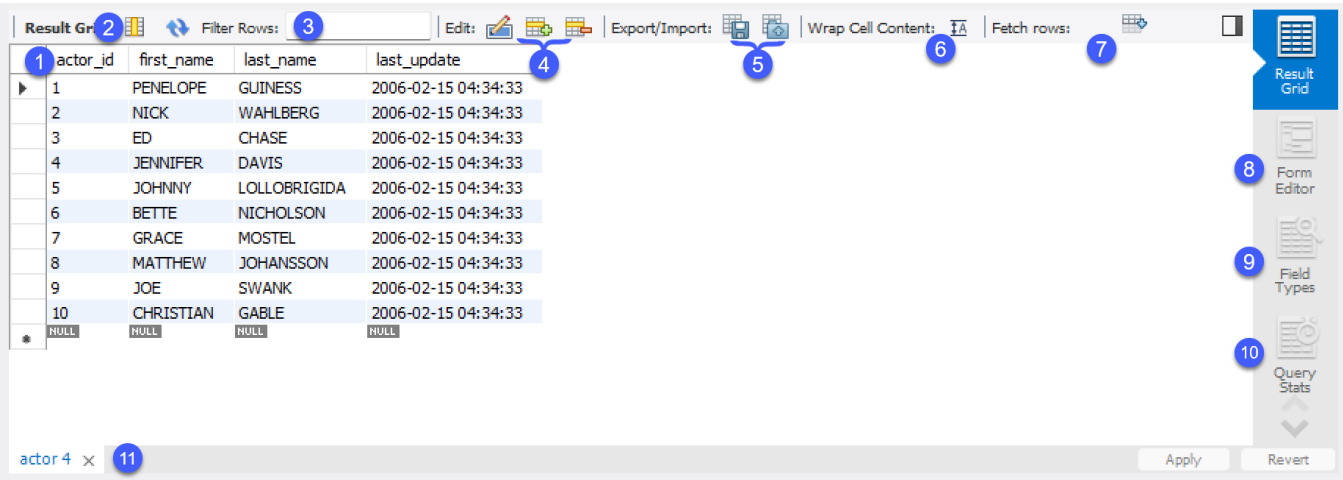
- There are several options for executing a query:
  - ◊ From the Query Editor window, use the toolbar icons
    - ∗ The **lightning bolt** executes either the selected portion of the query or, if nothing is selected, the entire query
    - ∗ The **lightning bolt + cursor** executes only the statement currently under the keyboard cursor
  - ◊ Using **keyboard shortcuts**
    - ∗ The shortcut Ctrl+Shift+Enter executes all or the selected portion of the query (same as the lightning bolt icon)
    - ∗ The shortcut Ctrl+Enter executes the statement currently under the cursor (same as the lightning bolt + cursor icon)
  - ◊ In addition to the same options already discussed, the Query menu at the top of the screen contains a few additional options for execution (not commonly used, but good to be aware of)
    - ∗ **Execute (All or Selection) to Text** displays results in a text-only format
    - ∗ **Execute Current Statement (Vertical Text Output)** displays the results of the current statement not as a table but by vertically listing all values for each row within separate text blocks
    - ∗ Both of these text output options can be copied from the results pane using right-click Select All + right-click Copy

# 1.C.7 Displaying Query Results

- Records returned by a query will display in a **Result Grid**

- Multiple actions can be taken within the Result Grid GUI:

  1. Sort by column
  2. Reset column sorting
  3. Filter rows by specified value
  4. Edit, insert, or delete a row
  5. Export or import records
  6. Turn on text wrapping
  7. Fetch next frame (where results are paginated)
  8. Edit records via fillable form
  9. View field attributes (data type, precision, etc.)
  10. View performance statistics for the query
  11. Navigate between result tabs



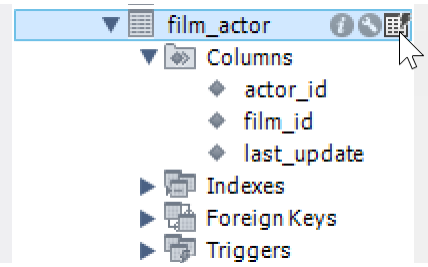
# 1.C.8 Exploring Schemas with Navigator

- Database objects can be explored using the Schemas tab of the Navigator panel

- Databases are listed at the top level

- Expanding the database allows you to explore:
  - ◊ Tables and their attributes (columns, indexes, foreign keys, triggers)
  - ◊ Other objects like Views, Stored Procedures, and User-defined Functions

- Hovering over a table name will reveal quick action buttons to display table details, modify table settings, or return records




- Clicking any object will display summary information about that object in the Information panel below the Navigator

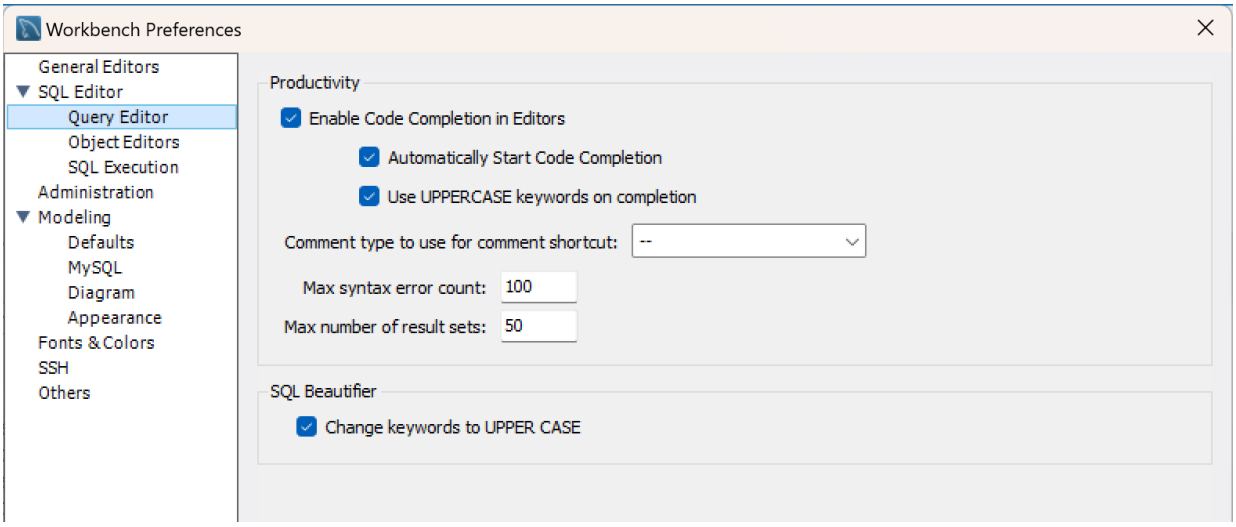
- Right-clicking any object will display menu options applicable to that object

- Dragging an object from the Navigator to the Query Editor will insert the name of the object wherever it is dropped in the query

# 1.C.9 Reviewing and Updating Preferences

- Workbench settings can be accessed from the Edit menu > **Preferences**

- One setting you might choose to update relates to capitalization of SQL keywords
  - ◊ Remember how we said that the common convention is to put keywords in all caps? MySQL Workbench's code auto-completion feature does not capitalize keywords by default, but you can change this in Preferences
  - ◊ In the Workbench Preferences settings, go to the Query Editor settings, then check the box next to **Use UPPERCASE keywords on completion**




- Some preference options may be limited
  - ◊ Fonts and font size are customizable for the query editor and results grid, but note that this does not affect other fonts in the GUI like menus or the output pane
  - ◊ Similarly, the color scheme may be changed to one of several preset options but not further customized

# Module 2

# Relational Database Design

# Section 2.A

# Understanding Normalization

# 2.A.1 Normalization

- **Normalization** is the formal process used to separate data into related tables within a relational database

- The goal of normalization is to reduce redundancy of data, meaning that each distinct piece of information is stored in only one place in the database (and then referenced elsewhere as needed)
  - ◊ This makes updating more efficient and makes better use of memory
  - ◊ Example:

**Before normalization**

| Title | Author | Author Country | Format | Price | Subject | Pages | Brand | Publisher Country |
|---|---|---|---|---|---|---|---|---|
| Beginning MySQL Database Design | Chad Ross | USA | Pamphlet | 49.99 | MySQL | 520 | Apress | USA |
| Beginning MySQL Database Design | Chad Ross | USA | Pamphlet | 49.99 | Databases | 520 | Apress | USA |
| Beginning MySQL Database Design | Chad Ross | USA | Pamphlet | 49.99 | Technical Design | 520 | Apress | USA |
| Beginning MySQL Database Design | Amar Patel | UK | Pamphlet | 49.99 | MySQL | 520 | Apress | USA |
| Beginning MySQL Database Design | Amar Patel | UK | Pamphlet | 49.99 | Databases | 520 | Apress | USA |
| Beginning MySQL Database Design | Amar Patel | UK | Pamphlet | 49.99 | Technical Design | 520 | Apress | USA |

**After normalization**

| BookID | Title | Format | Price | Pages | Brand | Publisher Country |
|---|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | Apress | USA |

| AuthorID | Author | Author Country |
|---|---|---|
| 1 | Chad Ross | USA |
| 2 | Amar Patel | UK |

| SubjectID | Subject |
|---|---|
| 1 | MySQL |
| 2 | Databases |
| 3 | Technical Design |

- Most databases are not “perfectly” normalized
  - ◊ Breaking normalization rules is called **denormalization**

# 2.A.2 Normalizing a Data Structure

- Normalization is achieved by applying a series of tests, called **normal forms**

- The normal forms are sequential, and each test requires that all the preceding normal form requirements are satisfied

- While there are a total of seven normal forms, generally a data structure is considered normalized if it follows the first three
  - These are known as the First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF)

# 2.A.3 First Normal Form (1NF)

- For a table to be in first normal form:
  - ◊ Each column in a table must contain only a single discrete value
  - ◊ Every row must be uniquely identifiable, typically by a primary key.
  - ◊ The table must not contain repeating columns for the same type of value, nor a list of multiple values crammed into one cell

- Let's look at an example:

| Title | Author | Author Country | Format | Price | Subject | Pages | Brand | Publisher Country |
|---|---|---|---|---|---|---|---|---|
| Beginning MySQL Database Design | Chad Ross, Amar Patel | USA, UK | Pamphlet | 49.99 | MySQL, Databases, Technical Design | 520 | Apress | USA |

How does this break 1NF?

- How about this example? What's the problem here?

| Order # | Customer | Item1 | Qty1 | Item2 | Qty2 | Total Quantity | Total Price |
|---|---|---|---|---|---|---|---|
| 365 | Jim Smith | Beginning MySQL Database Design | 1 | Python for Beginners | 1 | 2 | 99.98 |

# 2.A.4 Second Normal Form (2NF)

- Second normal form requires that every non-key column must depend on the entire primary key
  - ◊ "Entire" because may be a composite key
  - ◊ Example:
    In this version of an OrderDetails table, the composite key includes the Order # and ItemID, but the Item column only depends on the ItemID

composite key

| Order # (PK) | ItemID (PK) | Item | Qty |
|---|---|---|---|
| 365 | 1 | Beginning MySQL Database Design | 1 |
| 365 | 2 | Python for Beginners | 1 |

To apply 2NF, the Item column would need to be moved to a separate table (which would use ItemID as its primary key)

# 2.A.5 Second Normal Form, continued

- Each table should describe one entity, so splitting up composite entities (franken-entities) is also part of 2NF
  - ◊ Example: This table combines employees, authors, and customers, resulting in many null values for columns that don't apply to all roles, and requiring the addition of a Role column to differentiate types of person

| PersonID | Name | Role | Country | Salary | Royalty% | Phone |
|---|---|---|---|---|---|---|
| 11 | Chad Ross | Author | USA | null | 10 | 612-555-1234 |
| 12 | Amar Patel | Author | UK | null | 10 | 020 1234 5678 |
| 13 | Jim Smith | Customer | USA | null | null | 202-555-2345 |
| 14 | Lacy Vue | Sales | USA | 60000 | null | 414-555-4567 |

Splitting this into separate tables for Employee, Author, and Customer reduces unnecessary complexity and complies with 2NF:

| AuthorID | Name | Role | Country | Royalty% | Phone |
|---|---|---|---|---|---|
| 1 | Chad Ross | Author | USA | 10 | 612-555-1234 |
| 2 | Amar Patel | Author | UK | 10 | 020 1234 5678 |

| CustomerID | Name | Country | Phone |
|---|---|---|---|
| 1 | Jim Smith | USA | 202-555-2345 |

| EmployeeID | Name | Country | Salary | Phone |
|---|---|---|---|---|
| 1 | Lacy Vue | USA | 60000 | 414-555-4567 |

# 2.A.6 Third Normal Form (3NF)

- To comply with 3NF, every non-key column must depend *only* on the primary key

- A column that doesn't depend only on the primary key is either misplaced and belongs in another table, or it may be a computed value that can be calculated from other columns

  ◊ Example 1: The primary key for this table indicates that it should contain information specific to each product, but some of these details are really about the publisher, not the product

| ProductID | Title | Format | Price | Pages | Brand | Publisher Country | Publisher Contact | Publisher Phone |
|---|---|---|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | Apress | USA | Karl Bort | 630-555-9876 |
| 102 | Python for Beginners | Pamphlet | 49.99 | 430 | Crikey | UK | Ann Vang | 020 1234 8765 |

  Applying 3NF results in separate entities for Product and Publisher

| ProductID | Title | Format | Price | Pages | PublisherID |
|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | 1 |
| 102 | Python for Beginners | Pamphlet | 49.99 | 430 | 2 |

| PublisherID | Brand | Publisher Country | Publisher Contact | Publisher Phone |
|---|---|---|---|---|
| 1 | Apress | USA | Karl Bort | 630-555-9876 |
| 2 | Crikey | UK | Ann Vang | 020 1234 8765 |

  ◊ Example 2: How could this Order Detail table be brought into compliance with 3NF? (Hint: look for a calculated value)

| OrderDetailID | OrderID | ProductID | Qty | ItemPrice | ItemTotalPrice |
|---|---|---|---|---|---|
| 1 | 365 | 101 | 1 | 49.99 | 49.99 |
| 2 | 365 | 102 | 2 | 49.99 | 99.98 |

# 2.A.7 Adding Unique Row Identifiers

- By now you know that a table must have a primary key column that uniquely identifies each row in the table – but where does that unique identifier actually come from?
  - ◊ Often, the DBMS itself assigns that unique identifier each time a new row is added to the table, based on how the table is updated with new records

- If you are normalizing data – breaking one big table up into more specific tables – then you may be able to work with existing identifiers in your starting data, or you may need to assign an index column to a new table to create those identifiers
  - ◊ In the example on the previous page, the first table already includes an id for each product, but there is no id column for the brand/publisher

| ProductID | Title | Format | Price | Pages | Brand | Publisher Country | Publisher Contact | Publisher Phone |
|---|---|---|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | Apress | USA | Karl Bort | 630-555-9876 |
| 102 | Python for Beginners | Pamphlet | 49.99 | 430 | Crikey | UK | Ann Vang | 020 1234 8765 |

When the publisher info is broken out into a separate table, an id column is added to make a connection between the tables

| PublisherID | Brand | Publisher Country | Publisher Contact | Publisher Phone |
|---|---|---|---|---|
| 1 | Apress | USA | Karl Bort | 630-555-9876 |
| 2 | Crikey | UK | Ann Vang | 020 1234 8765 |

| ProductID | Title | Format | Price | Pages | PublisherID |
|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | 1 |
| 102 | Python for Beginners | Pamphlet | 49.99 | 430 | 2 |

  - ◊ The primary key does not have to use a number -- it just has to use a unique value for each row in the table

# 2.A.8 Responsible Denormalization

- Denormalization refers to intentionally going against normalization rules, and may be done for performance or to suit end-user needs
  - ◊ By separating data into many related tables, normalization reduces redundant storage – however, in order to retrieve useful information in queries, many `JOIN` statements may be required to assemble data from multiple tables
  - ◊ Many joins can slow down query processing
  - ◊ If the data that will frequently need to be joined is predictable, some denormalization may be desirable to improve database performance
- Some forms of denormalization are fairly common
  - ◊ For instance, tables with 1:1 relationships are often collapsed into a single table
  - ◊ Frequently used computed values might be added as column data, in order to mitigate slow query processing due to the multiple joins and aggregations required
    - * Note that a calculated value like this requires a mechanism to keep it up to date (trigger, stored procedure scheduled to run as a job)
- Denormalization is also common in data warehouse environments (designed to better enable analysis)

# Section 2.B

# Database Design Process

# 2.B.1 Database Design Process

- A database doesn't just spontaneously organize itself – it has to be intentionally and thoughtfully designed

- The design process typically includes multiple steps

- First, a plain-language description of business requirements is created, called the "conceptual model"

- Next, the conceptual model is translated into entities and relationships, producing the "logical model"
  - ◊ An entity is the precursor to what will become a table
  - ◊ Entities are normalized (the process of determining which pieces of data belong to which entity) to eliminate redundant data storage

- Third, tables are defined, including data types and constraints, based on the RDBMS being used, resulting in the "physical model" or schema

- Finally, the physical model is implemented using SQL `CREATE` statements to create the database and the tables it contains, into which data can then be populated

# 2.B.2 Model and Schema

- The words “model” and “schema” can have different shades of meaning depending on what type of database and what DBMS is being discussed
  - ◊ If you think that this seems confusing, you’re right! Just pay attention to how terms are being used in the current context, look up definitions in the relevant DBMS documentation, and *ask questions* to clarify meaning

- Generally, a **model** refers to the abstract design and structure of a database – how data is logically organized, the relationships between different data entities, and the rules governing the data
  - ◊ The model is like an architect's blueprint, outlining the design and structure of a building
  - ◊ When we get into using Microsoft Power BI, the term “model” will show up again, where a semantic model can include data from multiple sources

- Generally, a **schema** is the specific implementation of a model, setting out the structure of the database including definition of tables, columns, data types, indexes, constraints, and other database objects
  - ◊ The schema is like the actual construction plan, detailing the materials, dimensions, and procedures to build it
  - ◊ A schema is specific to a particular database

# 2.B.3 Creating the Conceptual Model

- The first step in designing a database (as with any tech project!) is the **gathering requirements**
  - ◊ **Requirements** refer to what the project needs to accomplish
  - ◊ Once you understand requirements, then you can start to consider how to meet those requirements
    - ∗ Example: "User must be able to access detailed information about customer purchase history" is a requirement
  - ◊ In the case of a database, this means **understanding what data will be stored, who/what will need access to it, and for what purposes**
- Requirements may include internal or back-end capabilities, or external or customer-facing capabilities
  - ◊ The **end-user** is the person who will be directly using the database (or feature or application or report), whether that is an internal user like a database administrator or external user like a customer
  - ◊ Often requirements focus on the needs of the end-user, although there may also be requirements for meeting business or technical needs
- Requirements can be derived from a wide variety of sources, including interviews with end-users, discussion with stakeholders or management, and researching prior existing solutions
  - ◊ For databases, usually this process begins to resolve into a list of nouns which identify entities that the database will need to store
  - ◊ The result of this process is the **conceptual model**

# 2.B.4 Creating the Logical Model

- The conceptual model is the starting point for creating the **logical model** which includes entities, attributes, relationships, and normalization
  - ◊ **Entities** represent things about which information is stored, which eventually will start to translate into tables
    - ∗ Examples: products, customers, orders
  - ◊ **Attributes** are individual facts about entities, which eventually will translate into columns and also should include a chosen entity identifier or primary key
    - ∗ Examples: product name, product description, cost, price
  - ◊ **Relationships** refer to how entities relate to one another or how information in one table connects to information in another
    - ∗ Example: the “orders” entity would likely have a relationship with the “customers” entity and also with the “products” entity
  - ◊ **Normalization** is the elimination of redundancy by placing only the information needed within each entity; goal is to eliminate storing the same piece of data in multiple places which could create problems as data is updated over time
    - ∗ Entities are each tested against a set of normalization rules (the “normal forms”) to ensure compliance

# 2.B.5 Relationship Considerations

- Relationships in relational databases have two major properties, cardinality and existence

- **Cardinality** describes how many records in one entity relate to how many records in the related entity
  - ◊ The three possible ways that a relationship may exist between two tables:
    - ∗ **One-to-many** (or many-to-one) – most common
    - ∗ **One-to-one** – less common
    - ∗ **Many-to-many** – e.g. one Customer can purchase multiple Products, but also one Product can be purchased by multiple Customers; this type of relationship must be resolved using a third table in between called an “associative” or “resolving” table

- **Existence** refers to whether a record is required on either side of a relationship – that is, whether the relationship is mandatory or not
  - ◊ For instance, can an Order exist without connecting to a Customer? Or can a Customer record exist without having placed any Orders?

# 2.B.6 Resolving Many-to-Many Relationships

- When two tables have a many-to-many relationship, a **resolving table** (or associative table or bridge table) is required
  - ◊ For instance, a product may be stocked in multiple stores, and a store will stock many different products

- A resolving table is made up of two foreign keys pointing back to each of the main tables, as well as any multi-dependency columns that may be needed
  - ◊ Example:

Product

| ProductID (PK) | ProductName | Price |
|---|---|---|
| P01 | Milk | $3.49 |
| P02 | Bread | $2.99 |
| P03 | Eggs | $4.29 |

Store

| StoreID (PK) | StoreName | City |
|---|---|---|
| S01 | Main St Market | Springfield |
| S02 | Riverside Grocers | Shelbyville |
| S03 | Downtown Fresh | Capital City |

Product_Store

| ProductID (PK/FK) | StoreID (PK/FK) | QuantityOnHand |
|---|---|---|
| P01 | S01 | 42 |
| P01 | S02 | 17 |
| P01 | S03 | 0 |
| P02 | S01 | 28 |
| P02 | S03 | 55 |
| P03 | S02 | 12 |

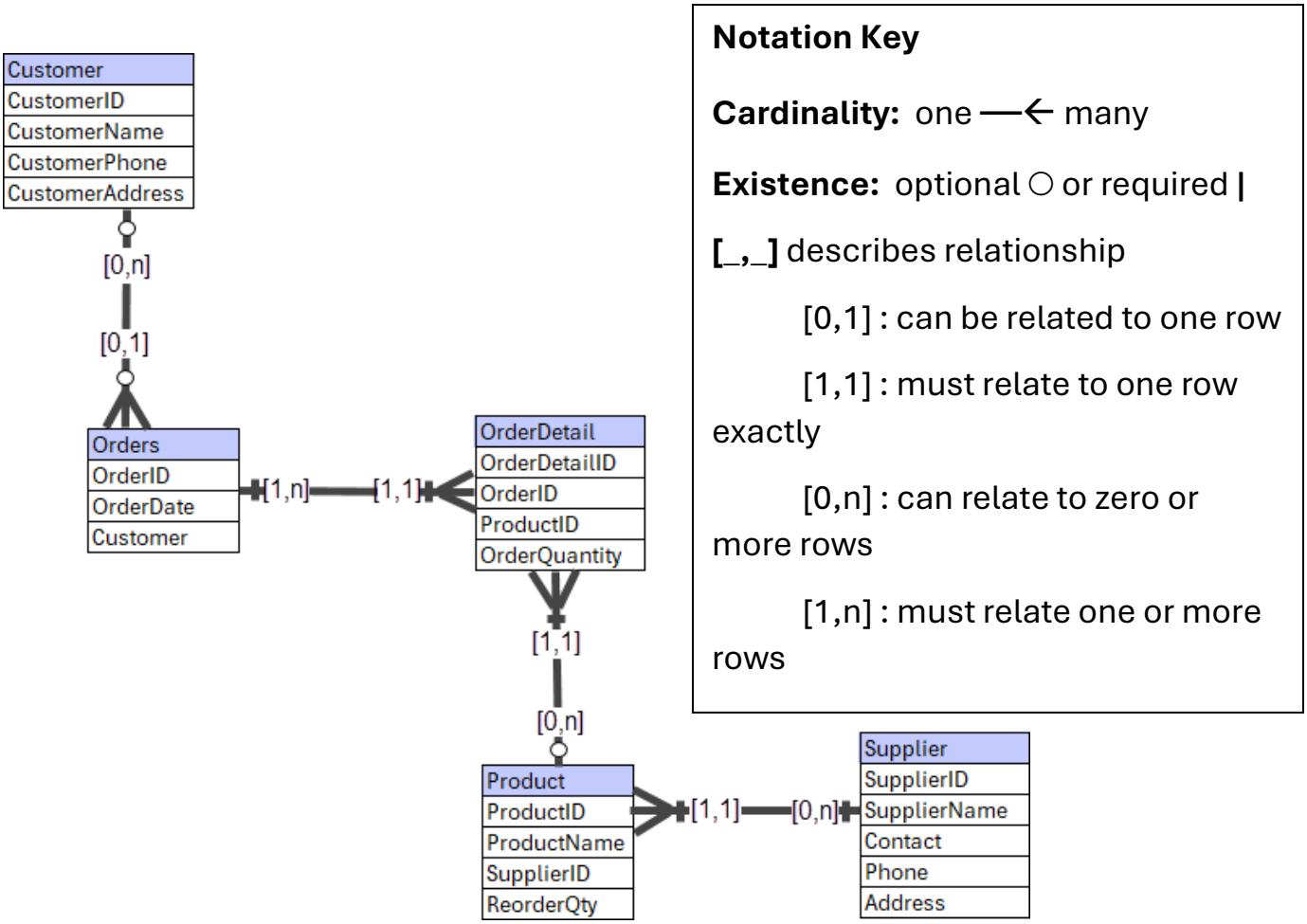
    - ∗ Notice that in the resolving table, ProductID and StoreID together make up the primary key (as a composite key), and separately each is a foreign key connecting to the other two tables

# 2.B.7 Documenting the Logical Model

- **Entity relationship diagrams (ERDs)** are used to document the logical model by illustrating its structure

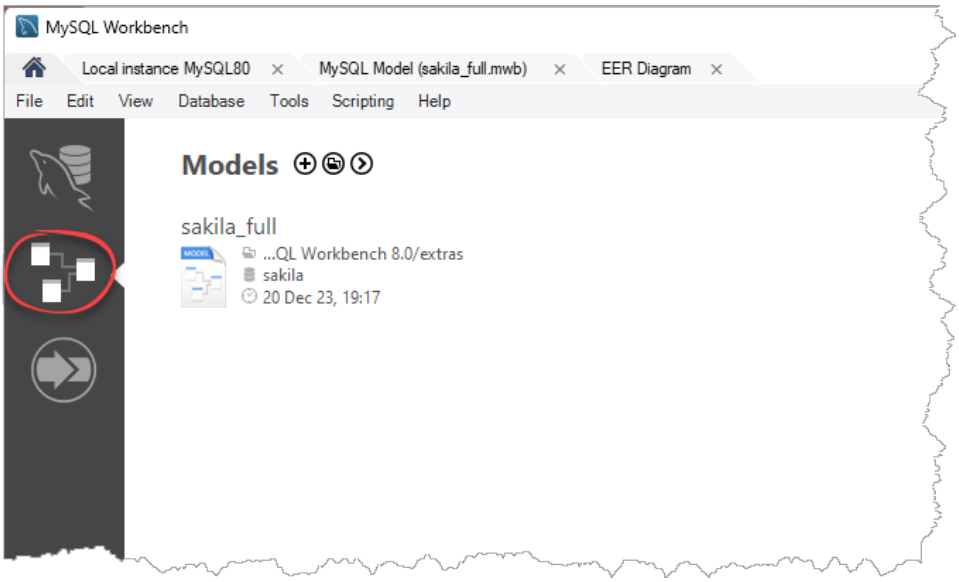
- Each entity is represented as a box, and attributes are listed for each entity
  - ◊ Remember, an entity is a possible future table, and its attributes are possible table columns

- Lines are drawn between each box to indicate relationships
  - ◊ Each line indicates both cardinality and existence requirements for the relationship



# 2.B.8 ER Diagram Tools

- In MySQL Workbench, the Model view allows you to create and access database models and ER diagrams




  - ◊ Clicking on a database will open tabs for the MySQL Model and EER Diagram (Enhanced Entity Relationship) for that database
  - ◊ Clicking the **+** icon next to models allows you to start a new model from scratch, while the **>** icon can be used to launch an EER generation workflow for an existing database

- There are similar tools available for other DBMS as well (Erwin, Oracle Data Modeler, TOAD Data Modeler…), sometimes called CASE (Computer Aided Software Engineering) tools

- Another popular online tool for creating entity relationship and other diagrams is https://app.diagrams.net (also called Draw.io)

# 2.B.9 Creating the Physical Model

- Using the logical model, the next design phase is to create the **physical model**

- The physical model describes the database structures that will be used to implement the logical model, using the desired RDBMS

- We've talked about converting entities to tables and attributes to columns, but there's a little more to it as well:
  - ◊ Implementing primary keys for each table
  - ◊ Using foreign keys to establish relationships based on the logical model
  - ◊ Setting column definitions including whether it will accept `NULL` values and what data type the column will use
  - ◊ Defining constraints and triggers to implement business logic
  - ◊ Determining handling of attributes that change over time, i.e. whether old values need to be retained or whether new values just overwrite the old

# 2.B.10 Physical Database Files

- What does it mean that databases have both a logical and physical structure?
  - ◊ The logical structure is the structure of tables and relationships
  - ◊ The **physical structure** is the storage location(s) on a disk subsystem (such as the hard drive)
    - ∗ At minimum, the physical structure requires a **data file** and a **transaction log file**
- Data files represent the storage location for all data related to a database
  - ◊ Includes records in tables and metadata in system tables
  - ◊ Most platforms create these files at the time the database is created
  - ◊ Additional data files can be created (sometimes called secondary data files)
    - ∗ These may be used to store databases on more than one physical disk or to take advantage of threaded processing on multiple CPUs
- The transaction log file stores a record of all changes to a database
  - ◊ Transactions are recorded sequentially
  - ◊ Changes are either **committed** or **rolled back** (not committed)
  - ◊ In multi-user environments, the log acts as a buffer for changes that need to be made to the database

# 2.B.11 Review of the Design Process

- First, the **conceptual model** sets forth all business requirements
- This is used to create the **logical model**, identifying *what* is to be stored in the database (entities, attributes, relationships)
  - ◊ The logical model may be embodied in an **entity relationship diagram**
- Next the **physical model** is created, defining *how* to store the data in the chosen RDBMS
  - ◊ Entities are translated into tables, and attributes into columns
    - * Columns are assigned names, data types, and whether `NULL` values are permitted
  - ◊ Primary keys are chosen for each table, and foreign keys are used to establish relationships between tables
  - ◊ Tables go through normalization, checking compliance with the first three normal forms (1NF, 2NF, and 3NF)
  - ◊ Resolving tables are created to bridge any many-to-many relationships
  - ◊ The end result of this process could be a diagram or a set of DDL scripts
- Finally, the physical database is built using the RDBMS via DDL (data definition language) scripts or using the platform's graphical interface

# Module 3

# Working with Tables

# Section 3.A

# Designing Tables

# 3.A.1 Choosing a Primary Key

- Each table needs to have a **primary key** that is used to uniquely identify records

- There are three possible types of primary key:
  - ◊ **Surrogate** (or autonumber or identity column)
    - ∗ An attribute added to the data which does not serve any purpose other than to be a unique identifier – very common
    - ∗ Example: auto-incrementing an Order ID number as orders are logged
  - ◊ **Natural** (or inherent)
    - ∗ Data includes a naturally occurring unique identifier
    - ∗ Example might be Social Security Number for credit reporting
    - ∗ Note that picking a natural key can be tricky, as it's sometimes hard to know for sure that an identifier will be unique for all records – for instance, phone numbers *seem* unique, but they also may change and even get reassigned to new customers
  - ◊ **Composite** (or multi-valued)
    - ∗ Combines two or more attributes of the existing data to produce a value that uniquely identifies each row
    - ∗ Example: composite of StoreID and ProductID in tracking inventory

- For a designated primary key, the requirement of uniqueness is enforced by the RDBMS

# 3.A.2 Foreign Keys

- **Foreign keys** are used to implement the relationships that were defined in the logical model

- A foreign key is a column in one table that references a record in another table's primary key – used to match records between them

- When a column is designated as a foreign key, values entered in the foreign key column must *already exist* in the primary key column
  - ◊ As with primary keys, the RDBMS enforces this restriction

- Example:
  - ◊ PublisherID is a foreign key in the first table and primary key in the second

Product

| ProductID | Title | Format | Price | Pages | PublisherID |
|---|---|---|---|---|---|
| 101 | Beginning MySQL Database Design | Pamphlet | 49.99 | 520 | 1 |
| 102 | Python for Beginners | Pamphlet | 49.99 | 430 | 2 |

Publisher

| PublisherID | Brand | Publisher Country | Publisher Contact | Publisher Phone |
|---|---|---|---|---|
| 1 | Apress | USA | Karl Bort | 630-555-9876 |
| 2 | Crikey | UK | Ann Vang | 020 1234 8765 |

If this company begins to order books from a new publisher, a new record must first be added in the Publisher table before any products for that publisher can be added to the Product table

# 3.A.3 Relationships: Cardinality

- Relationships between tables have the characteristics of *cardinality* and *existence*

- **Cardinality** between two tables can always be described in one of three ways:
  - ◊ **One-to-many**
    - Each record in the first table may correspond to many records in the second table (most common)
    - This may also be expressed as **many-to-one** depending on the order of tables
  - ◊ **One-to-one**
    - Each record in the first table corresponds to only one record in the second table
    - This is uncommon because such information can easily be combined into one table, however it may be used to avoid many null values when only a subset of records require additional attributes
  - ◊ **Many-to-many**
    - Many records in the first table may correspond with many records in the second table
    - This should be uncommon in a well-designed database as many-to-many relationships require a resolving table to function properly in a relational database

# 3.A.4 Relationships: Existence

- Apart from cardinality, **existence** refers to whether a record is *required* on either side of a relationship

- When designing data structures an architect needs to think carefully about all scenarios the data may need to support

- Example:
  - Is it required that an order has a customer?
    - In an e-commerce system, the answer is probably yes
    - In a point-of-sale system, the answer is probably no
    - In a system designed to handle both guest checkouts and registered customers, the compromise would need to be no
  - Is it required that a customer has an order?
    - Probably not immediately
    - But can you really call them a customer if they have never bought anything? (This is not just a rhetorical question, as data design decisions often reflect business rules)

# 3.A.5 Using Recursive Relationships

- A **recursive relationship** creates an association between one row of a table and another row in the *same* table

- Recursive relationships are relatively uncommon but can be another useful way to reduce redundancy of data stored in the database

- This type of relationship is also created using a foreign key, which has a distinct name but references the primary key of the same table
  - For example, in an Employee table containing records for each employee of a company, records might also reflect who each employee reports to, which will always be another employee in the same table
    - This could be implemented by creating a ReportsTo column as a foreign key which references the EmployeeID primary key

| EmployeeID (PK) | Name | Country | Role | ReportsTo (FK) |
|---|---|---|---|---|
| 1 | Lacy Vue | USA | Sales Representative | 2 |
| 2 | Minerva Blanc | USA | Sales Manager | 14 |

# 3.A.6 Defining Columns

- For all columns in a table, three things *must* be defined:
  - ◊ **Column name**
    - ∗ Should meaningfully identify the information stored in the field, e.g. ProductName
    - ∗ Ideally should be unique in the database, not ambiguous across tables, such as CustomerAddress versus just Address
    - ∗ Ideally should not include spaces – instead use PascalCasing or use_underscores
    - ∗ Typically a foreign key column should use the same name as the primary key it links to (except when creating recursive relationships)
  - ◊ **Data type**
    - ∗ Common data types include things like integers, decimals, text/varchar, and dates
    - ∗ One data type must be specified for the whole column
    - ∗ RDBMS will enforce the data type for all values entered, e.g. cannot enter text characters in a column with a numeric data type
  - ◊ **`NULL` or `NOT NULL` status**
    - ∗ Either a value is required for all values (`NOT NULL`), or `NULL` values are permitted in the column
    - ∗ Remember, `NULL` values are non-values and cannot be used directly in calculations or comparisons of data

# 3.A.7 More on Data Types

- Each column has one data type, which defines what type of data may be stored in that column

- Choose a data type that will fulfill the following guidelines:
  - ◊ Represent the full range of information required for the column
  - ◊ Minimize storage space
  - ◊ Minimize the amount of space used in memory when queries run

- Different DBMSs use different names for specific data types, but generally data types fall into these categories:
  - ◊ **Character string**: contains letters, numbers, and special characters
  - ◊ **Exact numeric**: integers (whole numbers) and decimal numbers
  - ◊ **Approximate numeric**: floating-point numbers, aka floats, useful for very large or very small numbers but subject to rounding
  - ◊ **Boolean**: TRUE or FALSE values
  - ◊ **Datetime**: date and time values
  - ◊ **Interval**: date and time intervals, e.g. a duration of 3 hours or 45 days
  - ◊ **Binary large object**: e.g. images, audio files, or documents stored directly in the database
  - ◊ There are also additional data types that do not fall into these categories, like JSON and other unique identifiers

# 3.A.8 String Data Types

- **String**, or “character string”, data types are used to represent text
  - ◊ A string is an ordered sequence of zero or more letters, numbers, spaces, or other characters
- String types can be specified as fixed or varying
  - ◊ Different DBMSs use different names for string data types and may impose different maximum lengths
- Values may be case sensitive depending on database collation settings
- When writing SQL statements, a string is always surrounded by single quotes
  - ◊ Example:

```
SELECT * FROM Products WHERE Format = 'Pamphlet';
```

  - ◊ If a string value contains an apostrophe, use two consecutive single quotes to represent it:
    - Entered as: `O''Meara`
    - Stored/displayed as: O'Meara

# 3.A.9 String Data Types in MySQL

- String types in MySQL are `CHAR`, `VARCHAR`, `TEXT`, `BINARY`, `VARBINARY`, `BLOB`, `ENUM`, and `SET`
  - ◊ **`CHAR`** (character) and **`VARCHAR`** (varying character) are the most commonly used string types and must be declared with a maximum length
    - * For example, `CHAR(30)` and `VARCHAR(30)` can each hold up to 30 characters
    - * The difference is that `VARCHAR` stores the string as-is, while `CHAR` pads shorter strings with spaces to fill the declared length
    - * `VARCHAR` is preferred today, but you may encounter `CHAR` if using legacy systems
  - ◊ **`TEXT`** holds a variable amount of character data similar to `VARCHAR`, but without declaring a length — useful for longer content like descriptions or notes
  - ◊ **`BINARY`**, **`VARBINARY`**, and **`BLOB`** are similar to the above but store files, images, and other non-text content rather than character strings
  - ◊ **`ENUM`** (short for "enumerated") and **`SET`** allow values to be restricted to a predefined list — very useful but less commonly encountered in introductory work

- For detailed documentation on MySQL string data types, see https://dev.mysql.com/doc/refman/8.0/en/string-types.html

# 3.A.10 Exact Numeric Data Types

- **Exact numeric** data types represent number values with a fixed precision and scale, and can be positive, negative, or zero.
  - ◊ **Precision** is the number of significant digits used to express the number (total number of digits to the left and right of the decimal point)
  - ◊ **Scale** is specifically the number of digits to the right of the decimal point
  - ◊ For example, the number 123.45 has a precision of 5 and a scale of 2

- MySQL exact numeric data types are `INTEGER`, `TINYINT`, `SMALLINT`, `MEDIUMINT`, `BIGINT`, `DECIMAL`, and `NUMERIC`
  - ◊ **`INTEGER`** or **`INT`** is a non-decimal number with a storage size of 4 bytes
    - * **Signed** integers can be negative or positive (up to about 2 billion or down to negative 2 billion); **unsigned** integers must be positive only (and therefore can store up to about 4 billion)
    - * **`TINYINT`**, **`SMALLINT`**, **`MEDIUMINT`**, **`BIGINT`** are similar to `INT` but vary in storage size (1, 2, 3, or 8 bytes)
  - ◊ **`DECIMAL`** stores exact numeric data values and is used when it is important to preserve exact precision, such as with monetary data
    - * When using `DECIMAL`, the precision and scale usually is specified, e.g. `DECIMAL(5,2)` to store values from -999.99 to 999.99
    - * Other versions of SQL also use a `NUMERIC` type which is functionally equivalent to `DECIMAL` in MySQL

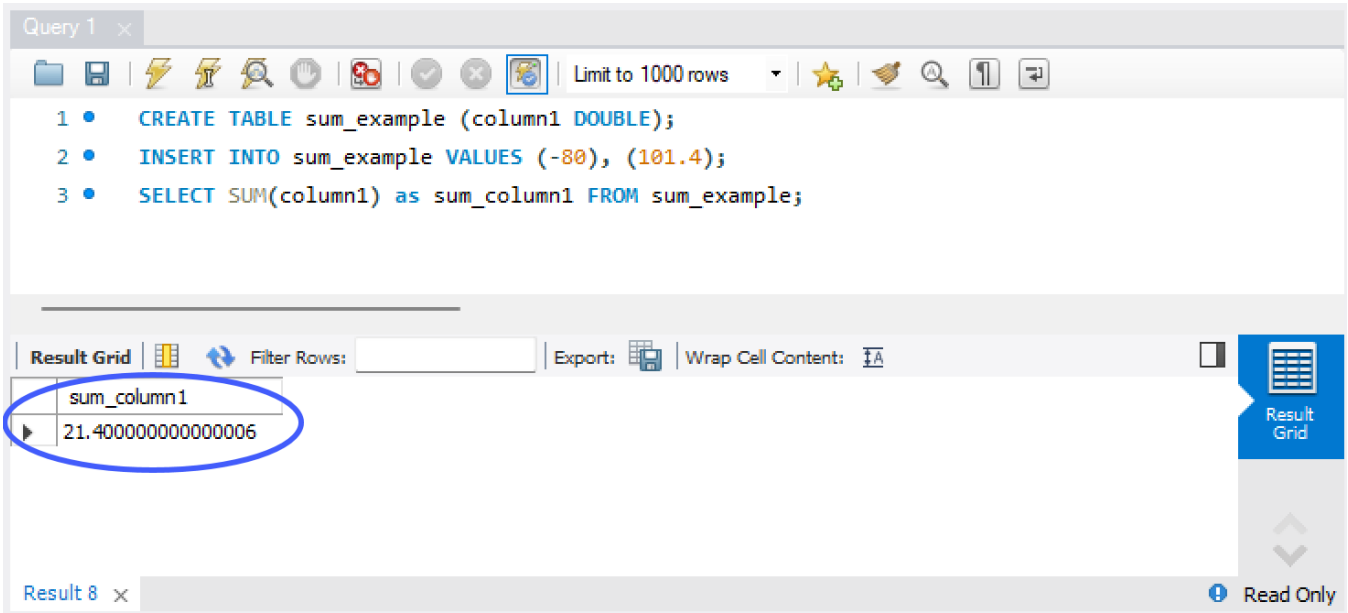
- For detailed documentation on MySQL exact numeric data types, see: [https://dev.mysql.com/doc/refman/8.0/en/numeric-types.html](https://dev.mysql.com/doc/refman/8.0/en/numeric-types.html)

# 3.A.11 Approximate Numeric Data Types

- Approximate numeric values can also be negative, positive, or zero, but unlike exact numeric types, they do not guarantee perfect precision, which can sometimes cause unexpected results
  - ◊ For floating point values, precision can be specified but not scale
  - ◊ Because scale is not restricted, floats are very useful where range is more important than precision, such as very small or very large values in scientific and statistical calculations
    - ∗ Note: older MySQL syntax allowed scale to be specified for floats but this was deprecated in MySQL 8.0
- This can mean that attempts to treat floating-point values as exact in comparisons may occasionally lead to problems
- MySQL uses the approximate data types `FLOAT` and `DOUBLE`
  - ◊ **`FLOAT`** is stored in 4 bytes and is accurate to about 6-7 significant digits
  - ◊ **`DOUBLE`** (short for "double precision") is stored in 8 bytes and is accurate to about 14-15 significant digits
    - ∗ The less-used type `REAL` is a synonym for `DOUBLE` by default in MySQL

# 3.A.12 What is an Approximate Value?

- Floating point numbers sometimes cause confusion
  - ◊ For example, using a pen and paper or a calculator:
    `-80 + 101.4 = 21.4`
  - ◊ However, if these values are stored in a column using the `DOUBLE` data type, and you use an SQL statement to find the `SUM` of the two values, the calculated result will be something like 21.400000000000006




  - ◊ If stored as a FLOAT, the result is less precise (21.400001525878906)

- This has to do with the way that approximate numerical values are stored
  - ◊ While a floating point value is displayed as a decimal number, as it is stored the underlying value is expressed in scientific notation – that is, as a decimal number multiplied by a power of 10
  - ◊ Example: `3.555E2 = 3.555*10`$^2$ `= 355.5`
    *Notice the syntax: uppercase E is used to represent the power of 10*

# 3.A.13 Examples of Approximate Values

- `FLOAT` (4 bytes) is generally accurate to about 6-7 significant digits
  - ◊ That could look like: `1.234567`
  - ◊ But a `FLOAT` could also store: `0.00000000000000000000001234567`
  - ◊ And also: `1234567000000000000000000000000000000000000000.0`
    - ∗ This is what it means to say that floating point numbers are stored with precision but not scale – they can easily scale very small or very large!
    - ∗ Conversely, you could **not** store `12340000000000.00000000567` as a `FLOAT`
- `DOUBLE` (stored in 8 bytes) is accurate to about 14 significant digits
  - ◊ Example: `1.2345678901234`
  - ◊ And also: `0.00000000000000000000000000000012345678901234`
  - ◊ And also: `1234567890123400000000000000000000000000000000.0`
- By contrast, the exact numeric `DECIMAL` data type can store a value with a precision of 65 decimal places (with memory variable based on the declared precision)
  - ◊ Example: `0.12345678901234567890123456789012345678901234567901 2345679012345`
    - ∗ This means that the `DECIMAL` type can store numbers with much greater precision, while approximate numeric types can store less precise numbers that are much, much larger or much, much smaller

# 3.A.14 Boolean Type

- The **`BOOLEAN`** data type is used to store truth values
  - These values include `TRUE`, `FALSE`, and `UNKNOWN` (stored by most DBMS as `NULL`)
- Boolean values are useful to indicate "yes" or "no" sorts of values
  - For example, a Products table might contain a Discontinued column using the Boolean data type
    - For a given row, if the product is discontinued, the value is `TRUE`
    - If the product is not discontinued, then the value is `FALSE`
    - Note that these `TRUE` and `FALSE` values should *not* be enclosed in single quotes because these are not string values
- Side note: Boolean values are also returned by conditional logic such as using the `WHERE` clause in a `SELECT` statement (we'll talk more about this later!)

# 3.A.15 Datetime Data Types

- Datetime data types are used to store date and time values
  - ◊ Date values follow the rules of the Gregorian calendar
  - ◊ Time values are based on a 24 hour clock
  - ◊ Parts of a date are separated by a hyphen (-), while parts of time are separated by a colon (:)
  - ◊ Time values *as stored* are specified with respect to UTC (Coordinated Universal Time, formerly GMT, Greenwich Mean Time)
    - * SQL standards require that every session have a default offset from UTC that is used for the duration of the session; for example, during daylight saving time, -4 is the time zone offset for Eastern Time, -5 for Central Time, -6 for Mountain Time, and -7 for Pacific Time

- The MySQL date and time data types are :
  - ◊ **`DATE`** is used to store dates in the format YYYY-MM-DD
  - ◊ **`TIME`** stores times as HH:MM:SS (may specify fractional seconds)
  - ◊ **`TIMESTAMP`** stores a combination of `DATE` and `TIME`, separated by a space (e.g. 2024-03-15 14:30:00); stores values in UTC and converts to the session time zone on retrieval, and is used where time zone awareness matters, such as recording transaction records
  - ◊ **`DATETIME`** stores a combination of date and time similar to `TIMESTAMP` but as the literal value entered without time zone conversion; used to store a fixed point in time that should not shift with time zone changes, such as a scheduled event
  - ◊ **`YEAR`** stores a year and accepts a variety of formats, including 4-digit string or 4-digit number in the range 1901 to 2155

# 3.A.16 Unique Identifiers

- Unique identifiers are mechanisms used to automatically generate primary key values, ensuring each record can be uniquely identified
  - ◊ Identifiers can be unique within a table, via an auto-increment column (sometimes called autonumber depending on DBMS)
  - ◊ Universally unique identifiers (UUIDs) can also be used
    - ∗ A UUID is a large random number that is generated by the DBMS which is statistically likely to be entirely unique
    - ∗ The resulting value is a string of five hexadecimal numbers in aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee format
    - ∗ Example: 550e8400-e29b-41d4-a716-446655440000
- Unique identifiers are established during column definition
- In MySQL:
  - ◊ Where **`AUTO_INCREMENT`** is set as a column attribute, for each record added to the column, the value saved in this column increases by 1 by default; the column data type can use any integer type (e.g. `INT`, `TINYINT`, `BIGINT`)
    - ∗ If a record is later deleted, the auto-increment value does <u>not</u> reuse the deleted number
  - ◊ **`UUID`** can be used in conjunction with `VARCHAR`
    - ∗ The primary key column definition uses a data type of `VARCHAR(36)` since UUID values are always 36 characters (32 hexadecimal digits plus 4 hyphens)
    - ∗ The `UUID` keyword can then be used to insert values into the column

# 3.A.17 Other Data Types

- A given DBMS may include additional data types
- For instance, MySQL supports data types for **`spatial data`** (used to store geographic coordinates and shapes, useful in mapping applications)
- MySQL also supports a **`JSON`** data type (JSON being a common format for storing structured data), which has some advantages over storing JSON-format text using a string datatype
- For more information on data types supported in MySQL, see: https://dev.mysql.com/doc/refman/8.0/en/data-types.html

# 3.A.18 NULL Values

- A column may be defined to permit `NULL` values (default) or not permit them (using `NOT NULL` attribute)

- `NULL` indicates that no value is recorded
  - ◊ A `NULL` value may be unknown or unable to be determined
  - ◊ As mentioned previously, this is different from a zero value or blank entry such as an empty string
  - ◊ For example, a `NULL` in a Discount column means the discount is unknown, whereas a zero means the discount is known to be zero
  - ◊ Similarly, a `NULL` in a Name column means the name is unknown, while a blank string indicates that the name is intentionally left blank

- Because `NULL` is a non-value, `NULL`s cannot be directly used in calculations or comparisons of data – however, you can handle them in various ways
  - ◊ `IS NULL` and `IS NOT NULL` can be used in queries to filter or test for `NULL` values
  - ◊ `COALESCE` is a function that returns the first non-null value from a list of values, and can be used in calculations where a `NULL` would otherwise cause the whole calculation to return `NULL`
    - * e.g. `COALESCE(Discount,0)` would check the Discount column and return the actual discount value if not `NULL`, or return 0 if Discount is `NULL`; this could be used inside an expression to calculate a discounted price such as `Price - COALESCE(Discount, 0)`

# Section 3.B

# Creating and Modifying Tables

# 3.B.1 Data Definition Language (DDL)

- DDL is the part of SQL used to create, modify, and delete databases and database objects

- DDL statements begin with one of three keywords:
  - ◊ `CREATE` – creates an object
  - ◊ `ALTER` – modifies an object
  - ◊ `DROP` – deletes an object

- Most platform GUIs also provide “clickable” functionality equivalent writing DDL scripts

- DDL commands can vary by RDBMS

# 3.B.2 Creating a Table

- `CREATE TABLE` is used to create and define new table
  - ◊ Can also use the graphical interface
- New tables require at minimum:
  - ◊ Name of the table
  - ◊ Column names and definition
- Table constraints may also be specified
- Example:
  - ◊ To create a Product table with this structure…

| ProductID | Title | Format | Price | Pages | PublisherID |
|---|---|---|---|---|---|

…you could use this SQL statement:

```
CREATE TABLE Product
(  ProductID INT PRIMARY KEY AUTO_INCREMENT,
   Title VARCHAR(255) NOT NULL,
   Format VARCHAR(25) NOT NULL,
   Price DECIMAL(10,2) NOT NULL,
   Pages INT UNSIGNED,
   PublisherID VARCHAR(10) NOT NULL,

   CONSTRAINT fk1 FOREIGN KEY(PublisherID)
REFERENCES Publisher(PublisherID) ON DELETE RESTRICT
ON UPDATE CASCADE
);
```

# 3.B.3 Modifying a Table

- The `ALTER TABLE` statement is used to modify an existing table:
  - ◊ Add columns to a table
  - ◊ Drop columns from a table
  - ◊ Modify data types or `NULL`-status of columns
  - ◊ Impose or modify primary key or foreign key constraints
  - ◊ Add default values to columns
  - ◊ Add the unique constraint to columns (values must be unique)
  - ◊ Impose check constraints (`ON UPDATE` or `ON DELETE`)
- When using `ALTER TABLE`, the statement must include
  - ◊ Name of the table to be modified
  - ◊ Any changes being made to the columns structure, starting with `ADD`, `ALTER`, or `DROP`
- Example:
  - ◊ To add a column to the previous Product table example:

```
ALTER TABLE Product
ADD ProductDescription TEXT;
```

  - ◊ To modify a column's data type from `VARCHAR` to `INT UNSIGNED`:

```
ALTER TABLE Product
MODIFY COLUMN PublisherID INT UNSIGNED;
```

# 3.B.4 Using a Default Value

- A default value provides a standard value to include for a column if no other value is provided when the record is added to the table
  - ◊ Where a value is provided, the default value is not used
- Arguments required for a default value are:
  - ◊ Name of the column
  - ◊ Value to use as the default (must match column data type)
- In MySQL, the default value can be set at column definition or added later using an `ALTER TABLE` statement
  - ◊ Example on table creation (uses `DEFAULT` in definition):

```
CREATE TABLE Publisher
(  PublisherID VARCHAR(10) PRIMARY KEY,
   Brand VARCHAR(25) NOT NULL,
   PublisherCountry VARCHAR(10) NOT NULL,
   PublisherContact VARCHAR(25) DEFAULT 'unknown',
   PublisherPhone VARCHAR(20)
);
```

  - ◊ Example on update (uses `SET DEFAULT` when updating definition):

```
ALTER TABLE Product
MODIFY COLUMN Price SET DEFAULT 999.99;
```

- Other DBMS may treat updating a table to add a default value as `ADD CONSTRAINT`, so check DBMS documentation

# 3.B.5 Constraints

- **Constraints** are restrictions on columns
  - ◊ Can be added to columns during initial column definition using `CREATE TABLE` and including `CONSTRAINT` statement
  - ◊ Or can be added to existing columns using `ALTER TABLE` with `ADD CONSTRAINT`
- Constraints may include:
  - ◊ Setting a primary key or foreign key(s)
  - ◊ Setting default values for the column
  - ◊ Validation against conditions (for example, requiring a 5-digit zip code)
  - ◊ Requiring that column values must be unique (no duplicates)
- In the `CONSTRAINT` statement, the constraint is assigned a name and given **arguments** to supply the rule made by the constraint
  - ◊ Example of `CONSTRAINT` syntax:

```
ALTER TABLE TableName
ADD CONSTRAINT {ConstraintName} {PRIMARY KEY |
 FOREIGN KEY | DEFAULT | CHECK | UNIQUE}
 ConstraintArguments;
```

# 3.B.6 Constraint Names

- When a constraint is defined, it is assigned a name (or "symbol"), either explicitly in the SQL statement or generated by the DBMS
  - ◊ Some DBMS may require an explicit name
  - ◊ Typically a name is short and includes some relevance to the constraint

- In MySQL, the following examples would all work to create a constraint:
  - ◊ On initial column definition:

```
CREATE TABLE Author (
  AuthorID INT PRIMARY KEY AUTO_INCREMENT,
  AuthorFirstName VARCHAR(30) NOT NULL,
  AuthorLastName VARCHAR(30) NOT NULL,
  AuthorCountry TINYINT,
  AuthorPhone VARCHAR(25),
  CONSTRAINT FOREIGN KEY(AuthorCountry) REFERENCES
     Country(CountryID) ON UPDATE CASCADE ON DELETE
     SET NULL);
```

    You can use the `SHOW CREATE TABLE` command to see that the above constraint is assigned the generated symbol `author_ibfk_1`

  - ◊ On update:

```
ALTER TABLE Product
ADD CONSTRAINT UniqueTitle UNIQUE (Title);
```

    This example specifies the constraint name `UniqueTitle`

    - ∗ There's nothing stopping you from naming a constraint anything at all – `big_banana` or whatever – but it can get confusing to troubleshoot

# 3.B.7 Primary Key / Foreign Key Constraints

- When a column is designated a **primary key**:
  - ◊ It is a required field (cannot be `NULL`)
  - ◊ Each record must have a unique value

- When a column is designated as a **foreign key**:
  - ◊ Must include `REFERENCES` keyword to point toward the table and column referenced by the foreign key column, and any other constraints
    - ∗ Example (revisited from 3.B.2 Creating a Table):

```
CREATE TABLE Product
(   ProductID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255) NOT NULL,
    Format VARCHAR(25) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Pages INT UNSIGNED,
    PublisherID VARCHAR(10) NOT NULL,

    CONSTRAINT fk1 FOREIGN KEY(PublisherID)
REFERENCES Publisher(PublisherID) ON DELETE
RESTRICT ON UPDATE CASCADE
);
```

- The foreign key column is not required to have the same name as the referenced primary key column, but this is a common convention and is a helpful cue on retrieving / reading table data
  - ◊ Except in the case of recursive relationships, where a foreign key references the primary key within the same table – then the foreign key column name must be different

# 3.B.8 Using Auto-increment or Identity Columns

- A primary key column may also be defined as an **identity column** (sometimes called autonumber column)
  - ◊ This is also called a **surrogate key**, where new data is being added to the table to create the primary key

- In MySQL, the keyword used to generate a unique identity for each row is `AUTO_INCREMENT` (as we saw in 3.B.2)
  - ◊ Looking again at the same example:

```
CREATE TABLE Product
(  ProductID INT PRIMARY KEY AUTO_INCREMENT,
   Title VARCHAR(255) NOT NULL,
   Format VARCHAR(25) NOT NULL,
   Price DECIMAL(10,2) NOT NULL,
   Pages INT UNSIGNED,
   PublisherID VARCHAR(10) NOT NULL,

   CONSTRAINT fk1 FOREIGN KEY(PublisherID)
REFERENCES Publisher(PublisherID) ON DELETE RESTRICT
ON UPDATE CASCADE
);
```

  - ◊ To start with an `AUTO_INCREMENT` value other than 1, the starting value can be set with `CREATE TABLE` or `ALTER TABLE`, like this:

```
ALTER TABLE Product AUTO_INCREMENT = 100;
```

- To retrieve the most recently generated `AUTO_INCREMENT` value for a column, the `LAST_INSERT_ID()` function can be used

# 3.B.9 ON DELETE AND ON UPDATE Constraints

- The **ON DELETE** and **ON UPDATE** constraints dictate behavior of related tables where a foreign key in a "child" table references a primary key in a "parent" table

- `ON DELETE` options for MySQL (keywords may vary by DBMS):
  - ◊ `ON DELETE` **CASCADE**: If a value is deleted from the primary key in the parent table, any corresponding record in the child table will also be deleted
  - ◊ `ON DELETE` **SET NULL**: If the parent row is deleted, the child foreign key value will be set to `NULL` (column must allow `NULL` values)
  - ◊ `ON DELETE` **SET DEFAULT**: If the parent row is deleted, the child foreign key value will be set to the default value provided
  - ◊ `ON DELETE` **RESTRICT**: A row in the parent table cannot be deleted if there is a row in the child table referencing it

- `ON UPDATE` options follow the same pattern:
  - ◊ `ON UPDATE` **CASCADE**: If a value is updated in the parent table primary key, any corresponding record in the child table will also be updated
  - ◊ `ON UPDATE` **SET NULL**: If the parent row is changed, the child foreign key value will be set to `NULL` (column must allow `NULL` values)
  - ◊ `ON UPDATE` **SET DEFAULT**: If the parent row is changed, the child foreign key value will be set to the default value provided
  - ◊ `ON UPDATE` **RESTRICT**: A row in the parent table cannot be altered if there is a row in the child table referencing it

# 3.B.10 `Unique` Constraint

- The `UNIQUE` constraint requires that every record value for that column is unique, with no duplicates
  - ◊ A column designated as a primary key is by definition always unique
  - ◊ Other columns using the `UNIQUE` constraint may sometimes be referred to as candidate keys

- This constraint can be applied at column definition or added to an existing column
  - ◊ Example, adding the `UNIQUE` constraint to an existing Product column:

| ProductID | Title | Format | Price | Pages | PublisherID |
|---|---|---|---|---|---|

```
ALTER TABLE Product
ADD CONSTRAINT UniqueTitle UNIQUE (Title);
```

# 3.B.11 CHECK Constraints

- The `CHECK` constraint allows inserted data to be validated against a set condition referencing the same or another column and can be set at the table or column level
  - ◊ The condition must evaluate to a Boolean value (TRUE or FALSE)
  - ◊ Each time a record is inserted or updated, the changed value is tested against the `CHECK` condition
  - ◊ `CHECK` can reference the column it's applied to, other columns in the same table, or (via foreign key) columns in another table
  - ◊ Existing records default to being checked when the constraint is defined
  - ◊ This constraint can also be created but not used, or can be temporarily turned off, using `NOT ENFORCED`
    - ∗ Other DBMS may use the keyword `NOCHECK CONSTRAINT` (reapplied using `CHECK CONSTRAINT`)

- In MySQL, a check constraint can be set using CREATE TABLE or added/dropped/modified using an ALTER TABLE statement
  - ◊ To add a CHECK constraint within a column definition, use the syntax:

```
column_name datatype CHECK (expression)
```

  - ◊ To add at table level on table creation:

```
CONSTRAINT [symbol] CHECK (expr) [[NOT] ENFORCED]
```

  - ◊ To add/drop/modify a `CHECK` constraint using `ALTER TABLE`, preface the above syntax with `ADD`, `DROP`, or `ALTER`

# 3.B.12 Example: Check Constraints

- Examples adding, altering, and removing a check constraint
  - ◊ Adding `CHECK` constraint on column definition:

```
CREATE TABLE SalesRegion
(  SalesRegionID INT PRIMARY KEY AUTO_INCREMENT,
   SalesRegion VARCHAR(50) CHECK (SalesRegion >= 3)
     ENFORCED);
```

  - ◊ Note that when the above example is executed, the constraint will automatically be assigned the name `salesregion_chk_1` which we can see using `SHOW CREATE TABLE SalesRegion;`

```
+ ---------- + ---------------- +
| Table      | Create Table     |
+ ---------- + ---------------- +
| SalesRegion | CREATE TABLE `salesregion` (
  `SalesRegionID` int NOT NULL AUTO_INCREMENT,
  `SalesRegion` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`SalesRegionID`),
  CONSTRAINT `salesregion_chk_1` CHECK ((`SalesRegion` >= 3))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+ ---------- + ---------------- +
1 rows
```

  - ◊ To alter this constraint and change it to `NOT ENFORCED`:

```
ALTER TABLE SalesRegion
ALTER CHECK salesregion_chk_1 NOT ENFORCED;
```

# 3.B.13 Dropping a Constraint

- To drop a constraint, use `ALTER TABLE` with `DROP CONSTRAINT`:
  - ◊ Include the name of the table and name of the constraint
  - ◊ Example:

```
ALTER TABLE SalesRegion
DROP CONSTRAINT salesregion_chk_1;
```

  - ◊ A CHECK constraint can also be removed using DROP CHECK:

```
ALTER TABLE SalesRegion
DROP CHECK salesregion_chk_1;
```

# 3.B.14 Creating a Temporary Table

- Sometimes you might want to get a quick snapshot of data, test query modifications on a copy of the data, or have a holding place for results of complex queries that you are working on – without making it a permanent part of your database structure

- In these cases, you can use the `CREATE` keyword to generate a **temporary table** that will automatically be dropped by the DBMS at the end of your session
  - ◊ Different DBMS use different variations on how to create a temporary table
  - ◊ In MySQL, `CREATE TEMPORARY TABLE` is used (plus table name and column definitions)
    - * The user profile must also have the `CREATE TEMPORARY TABLES` privilege to create temporary tables

- A temporary table may sometimes also be generated by execution of a stored procedure
  - ◊ If the stored procedure carries the privileges of a user who does have `CREATE TEMPORARY TABLES` (a "definer-context stored procedure"), it can be executed by a user without that privilege to create a temporary table, with the lower-privilege user can then interact with and perform operations on

# 3.B.15 Dropping a Table

- If a table is no longer needed or was created by mistake, it can be removed using the DROP TABLE statement
  - ◊ Syntax

    ```
    DROP TABLE table_name;
    ```

  - ◊ MySQL Workbench also provides a GUI option by right-clicking a table name and selecting Drop Table
- Be aware that dependencies may prevent a table from being dropped, for instance if its primary key is being referenced as a foreign key by another table

# Module 4

# Working with Records

# Section 4.A

# Inserting, Updating, and Deleting Records

# 4.A.1 Introducing DML

- The part of SQL used to interact with records is known as Data Manipulation Language or DML

- DML statements begin with one of the following keywords:
  - INSERT – adds records to tables
  - UPDATE – modifies existing records
  - DELETE – removes records from tables
  - SELECT – retrieves / reads records from tables

- These types of statements often form the bulk of how a user interacts with a particular database
  - Database administrators use DML to query system tables and views which gives them information about the properties of a database
  - Developers and analysts use DML to search for and display useful data, as well as to insert or update records

# 4.A.2 `INSERT` Statements

- The `INSERT INTO` statement is used to add a new row (also called a record) to a table

- There are two possible techniques:
  - ◊ You can specify the table name and the values of the rows to be added
    - * Syntax:
      ```
      INSERT INTO table-name
      VALUES (value1, value2, value3, ...);
      ```
  - ◊ Or you can specify the table's column names and the values of the rows to be added
    - * Syntax:
      ```
      INSERT INTO table-name (column1, column2, ...)
      VALUES (value1, value2, ...);
      ```
    - * Using this approach, any columns you omit would have `NULL` values for that row (if a non-nullable column, the INSERT statement will fail)

# 4.A.3 `UPDATE` Statements

- Use the `UPDATE` statement to edit rows in a table
  - ◊ It allows you to change the value in one or more columns of a row
- Use the `WHERE` clause to specify which row(s) will have the changes made to them
  - ◊ Syntax

```
UPDATE table-name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

- Be careful with the `UPDATE` statement – if you forget the `WHERE` clause, all the records in that table will be updated!
- MySQL will generate an error if the update condition isn't based on a primary key
  - ◊ You can turn that off with the following code

```
SET SQL_SAFE_UPDATES=0;
```

  - ◊ Turn the safety setting back on again after your query

```
SET SQL_SAFE_UPDATES=1;
```

# 4.A.4 DELETE Statements

- To delete one or more rows from a table, use the `DELETE` statement
  - ◊ Syntax

```
DELETE FROM table-name
WHERE condition;
```

- Deleting a record is a precarious action because the row you want to delete may be referenced in other tables
  - ◊ For example, if you try to delete a specific film from the Film table, you will now have a reference in Inventory to a film_id that doesn't exist
- NOTE: Many companies don't actually delete records... Instead, they have a column whose name might be "Active" or something similar
  - ◊ You set Active to `TRUE` when the row is created and mark it "deleted" by using the `UPDATE` statement to set active to `FALSE`
  - ◊ This will allow the database to maintain referential integrity between related tables

# 4.A.5 `SELECT` Statements

- The **`SELECT`** statement is used to **select** / retrieve data from a table in a database
  - ◊ Basic syntax:

    ```
    SELECT [what] FROM table;
    ```

    - `SELECT *` retrieves all columns
    - To only retrieve certain columns, specify by column name as `SELECT column1, column2` [, etc.]
  - ◊ `WHERE` is used to specify any conditions that records must match
  - ◊ Other optional clauses include `JOIN`, `ORDER BY`, and more (we'll talk about these next week…)

- Note that `SELECT` can also be used to return the result of an expression
  - ◊ Example:

    ```
    SELECT 85 + 33;
    SELECT 'Do not delete';
    ```

# 4.A.6 `WHERE` Clauses

- The `WHERE` clause specifies criteria / condition that records must meet to be affected by the query
  - ◊ Used with `SELECT` to specify criteria that records must match in order to be returned
  - ◊ Used with `UPDATE` and `DELETE` statements to indicate which records to modify or delete – and this is really important!
    - ∗ Always, always use `WHERE` whenever using `UPDATE` or `DELETE` or you will make the same change to *every single record in the database*
- `WHERE` requires three arguments:
  - ◊ The name of the column to be matched against
  - ◊ A comparison operator (`=, <, <=, >, >=, <>, !=,` or `LIKE`)
  - ◊ A value to compare
  - ◊ Syntax:

```
WHERE column_name [operator] value;
```

- Also permits multiple criteria to be specified with `AND` or `OR`

# Section 4.B

# Error Handling

# 4.B.1 Brief Intro to Error Handling Concepts

- When writing SQL statements, **errors** and unexpected results are inevitable — knowing how to respond to them is an important part of working with databases

- There are broadly two categories of problems to be aware of:
  - **Syntax errors:** the query is not written correctly and the RDBMS cannot execute it
    - MySQL Workbench will highlight these and return an error message describing the problem and the line where it occurred
    - Reading error messages carefully is the first and most important debugging skill
  - **Logic errors:** the query runs successfully but returns unexpected or incorrect results
    - These are often harder to catch than syntax errors because the database doesn't know your intent
    - Always sanity-check your results, especially after `UPDATE` or `DELETE` statements

# 4.B.2 Testing for Errors with SELECT

- For `INSERT`, `UPDATE`, and `DELETE` statements in particular, it is good practice to test your `WHERE` condition with a `SELECT` statement first before executing the change
  - ◊ Example:

    Before deleting records, first run
    ```
    SELECT * FROM orders
    WHERE order_date < '2020-01-01';
    ```
    to confirm you are targeting the right records

    If the results are what you want to delete, *then* run
    ```
    DELETE FROM orders
    WHERE order_date < '2020-01-01';
    ```

# 4.B.3 Using Transactions

- MySQL also supports **transactions**, which allow you to group one or more statements together so that they either all succeed or all fail together
  - ◊ A transaction is started with `BEGIN` and concluded with either `COMMIT` (to save the changes) or `ROLLBACK` (to undo them)
  - ◊ Example:
    ```
    BEGIN;
    DELETE FROM orders WHERE order_date < '2020-01-01';
    -- review results before committing
    ROLLBACK; -- or COMMIT; if results look correct
    ```
- Using transactions gives you a safety net when making changes to data, since ROLLBACK will undo everything back to the BEGIN statement
  - ◊ Note: In this course you are working with local sample data, so mistakes are easily recoverable, but understanding best practices now will serve you well when working with real production databases where mistakes can have significant consequences

# Section 4.C

# Batch Importing and Exporting

# 4.C.1 Importing existing data

- Records may be added to a database from an existing file, such as a spreadsheet or delimited text file

- MySQL Workbench includes a Table Data Import Wizard that helps streamline the process of importing data from a .csv or .json file

# 4.C.2 Exporting table data

- MySQL Workbench also includes a Table Data Export Wizard
  - ◊ From the Navigator panel, this utility allows you to quickly select columns from a table for export to .csv or .json
  - ◊ From the Results Grid, you can easily export the record set that is the result of a query

# 4.C.3 Importing an existing database

- An existing database can also be imported into an RDBMS, with schema, relationships, records and all
  - ◊ A database created using one RDBMS may or may not play nicely with a different RDBMS
  - ◊ This topic is really beyond the scope of our Data Analyst course to go into in detail – in a real-life scenario, you would likely be working on this with a database administrator or other IT professional at your organization

# End of Week 2 Student Guide